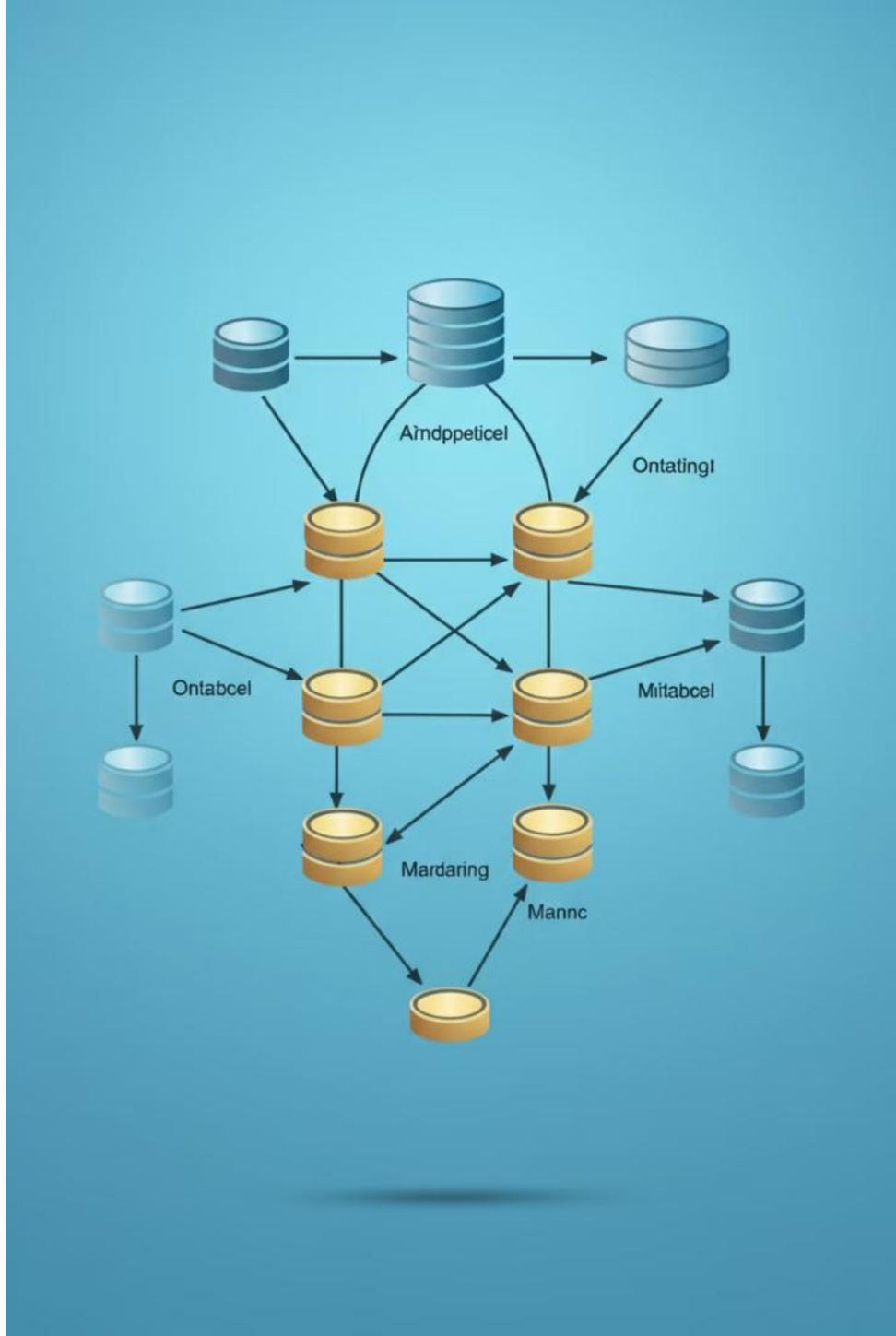
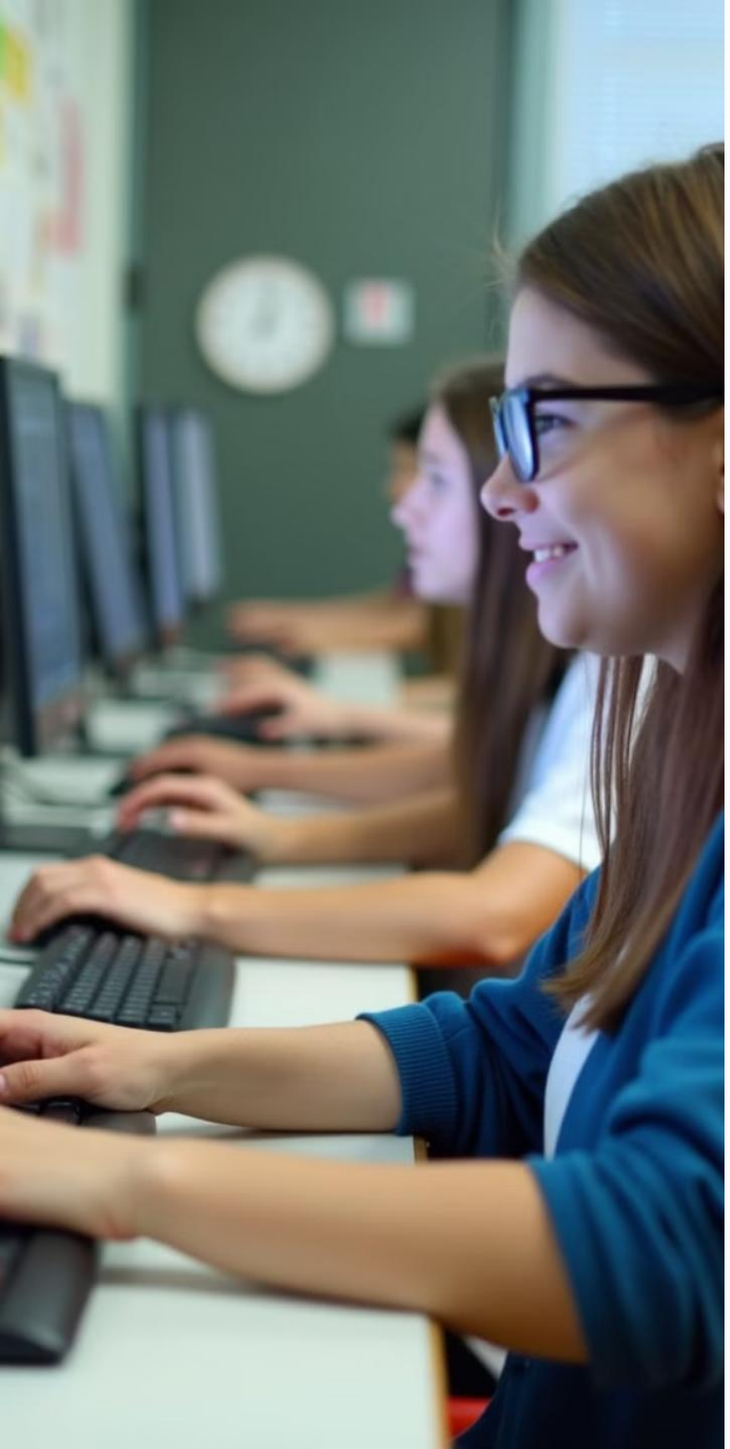


Módulo: Acceso a datos

UT3: ORM: Herramientas de mapeo objeto relacional





RESULTADO DE APRENDIZAJE: RA3

RA3. Gestiona la persistencia de los datos identificando herramientas de mapeo objeto relacional (ORM) y desarrollando aplicaciones que las utilizan.

CRITERIOS DE EVALUACIÓN		%
a)	Se ha instalado la herramienta ORM.	**1/7**
b)	Se ha configurado la herramienta ORM.	**1/7**
c)	Se han definido los ficheros de mapeo.	**1/7**
d)	Se han aplicado mecanismos de persistencia a los objetos.	**1/7**
e)	Se han desarrollado aplicaciones que modifican y recuperan objetos persistentes.	**1/7**
f)	Se han desarrollado aplicaciones que realizan consultas usando el lenguaje SQL.	**1/7**
g)	Se han gestionado las transacciones.	**1/7**



Índice de la UT3

1 **Introducción al mapeo objeto-relacional**

Origen y concepto ORM. Ventajas e inconvenientes

2 **Características y herramientas ORM**

Principales características. Herramientas más utilizadas. Hibernate y JPA.

3 **Instalación y configuración**

Instalación de Hibernate. Configuración con persistence.xml e hibernate.cfg.xml.

4 **Clases persistentes y mapeo**

Entidades y anotaciones. Generación desde la BD. Relaciones, colecciones y herencia. Ficheros .hbm.xml.

5 **Gestión de la persistencia**

EntityManager, contexto de persistencia, estados de una entidad y transacciones.



Índice de la UT3

6 **Operaciones de persistencia**

Almacenar, cargar, modificar y eliminar objetos. Cambios de estado.

7 **Consultas**

Consultas SQL nativas y JPQL. Cuándo usar cada una.

1. INTRODUCCIÓN

1.1. Historia y evolución de ORM

- El mapeo objeto-relacional (ORM) surgió como respuesta a la creciente popularidad de los **lenguajes orientados a objetos, como C++ y Java**, en las **décadas de 1980 y 1990**.

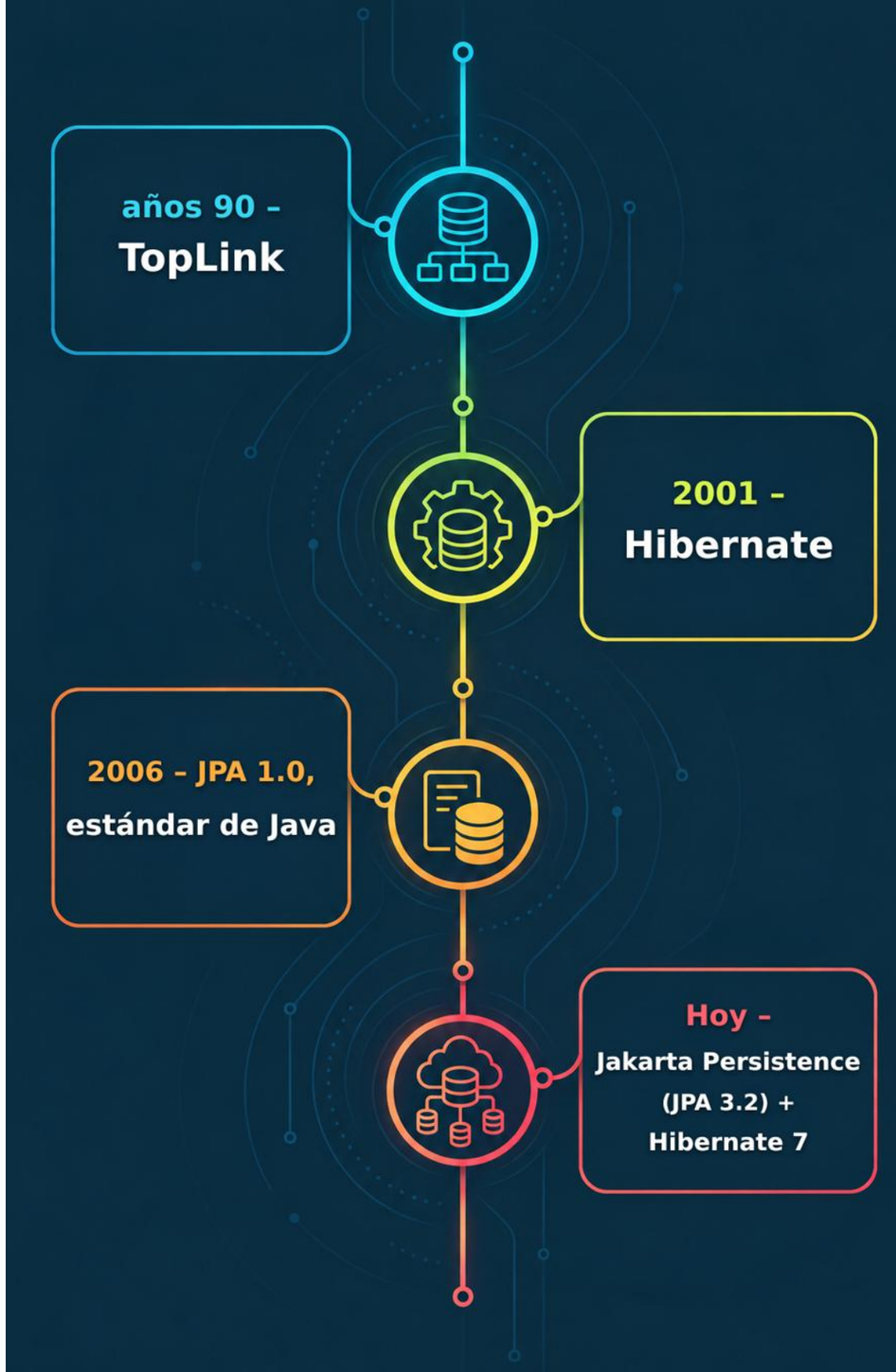
- Durante este tiempo, las bases de datos relacionales dominaron como el principal medio de almacenamiento de datos, lo que planteó la necesidad de integrar ambos paradigmas.

- La evolución culmina en un estándar: JPA (Jakarta Persistence), con implementaciones como Hibernate.





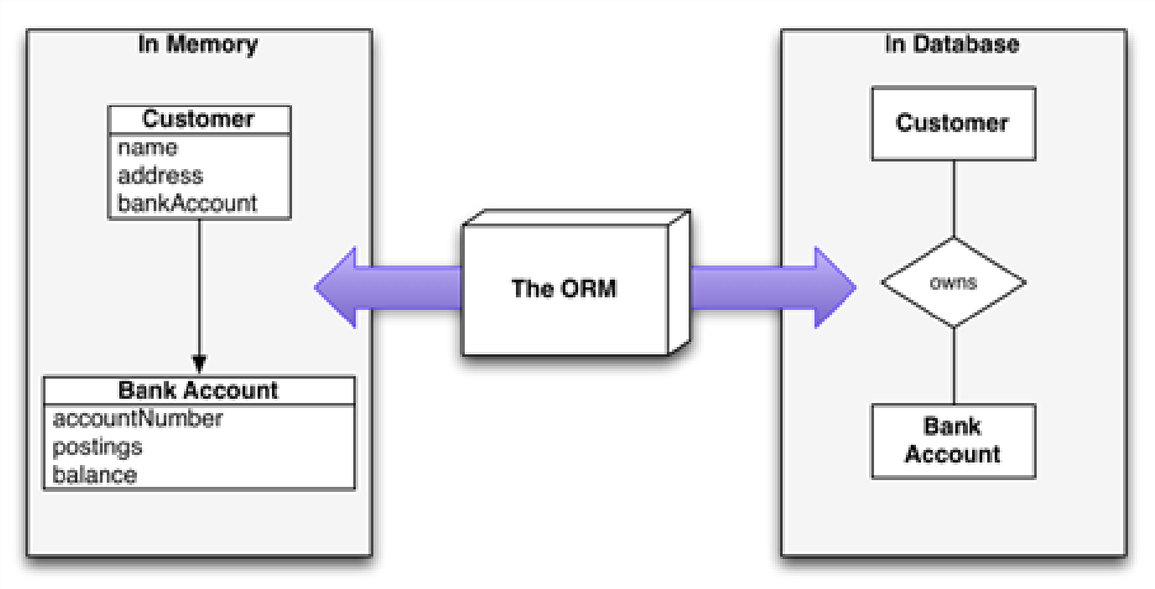




1. INTRODUCCIÓN

1.2. Concepto de ORM

- El mapeo objeto-relacional (Object-Relational Mapping, ORM) es una **técnica de programación** que permite **convertir datos** entre los sistemas de tipos utilizados en los lenguajes de **programación orientados a objetos** y los sistemas de tipos empleados en **bases de datos relacionales**.
- En esencia, el **ORM crea una base de datos orientada a objetos virtual sobre una base de datos relacional**.

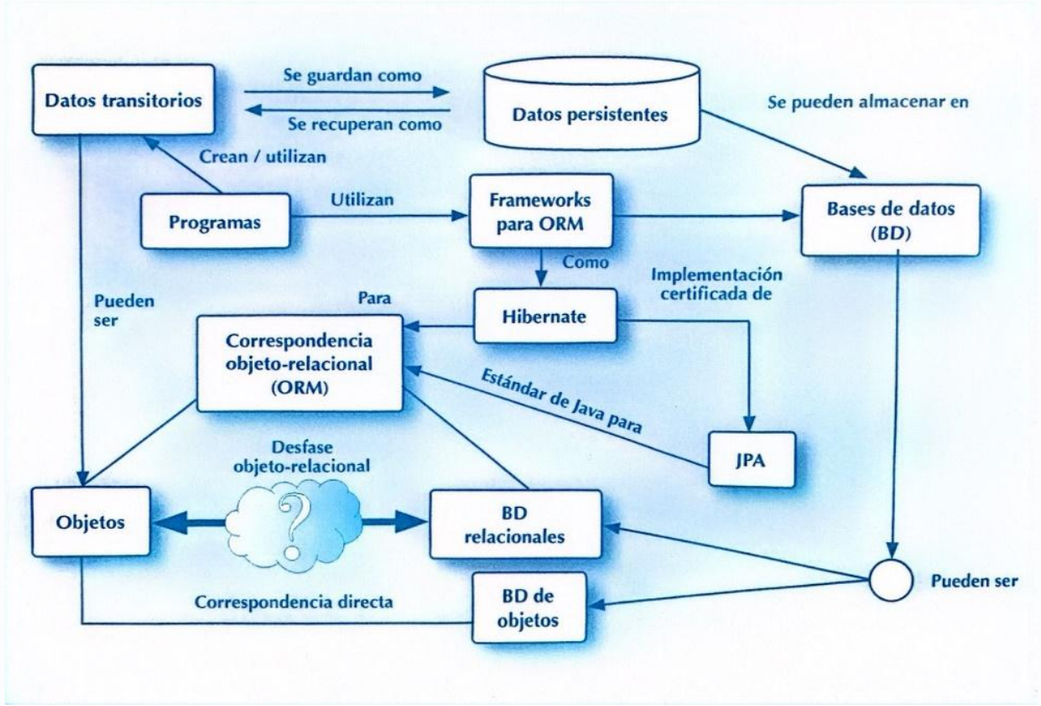


1. INTRODUCCIÓN

1.2. Concepto de ORM

- Resuelve el desfase objeto-relacional (UT2): los objetos tienen clases, herencia y referencias; las tablas, filas y claves foráneas.

- El ORM **traduce los datos de los objetos en datos primitivos almacenables en una base de datos relacional y viceversa**, permitiendo recuperar objetos a partir de datos almacenados.



1. INTRODUCCIÓN

1.3.1 Ventajas del ORM

1. Rapidez en el desarrollo

Las herramientas ORM reducen el código de acceso a datos y permiten generar automáticamente las clases a partir del esquema de la base de datos.

JDBC

```
. . .
String sentenciaSql = "INSERT INTO personajes (nombre, descripcion, vida, ataque)"
  ") VALUES (?, ?, ?, ?)";
PreparedStatement sentencia = conexion.prepareStatement(sentenciaSql);
sentencia.setString(1, personaje.getNombre());
sentencia.setString(2, personaje.getDescripcion());
sentencia.setInt(3, personaje.getVida());
sentencia.setInt(4, personaje.getAtaque());
sentencia.executeUpdate();

if (sentencia != null)
  sentencia.close();
. . .
```

ORM

```
em.getTransaction().begin();
em.persist(personaje);
em.getTransaction().commit();
```



1. INTRODUCCIÓN

1.3.1. Ventajas del ORM

2. Abstracción de la base de datos

Un sistema ORM abstrae la lógica del sistema gestor de bases de datos (SGBD). Si en el futuro se cambia de SGBD, el impacto en el código es mínimo o inexistente.

3. Reutilización

Los métodos asociados a los objetos de datos pueden ser reutilizados en diferentes partes de la aplicación, e incluso en aplicaciones distintas.

4. Mantenimiento del código

Al estructurar de forma ordenada la capa de datos, el ORM facilita el mantenimiento y la comprensión del código.



1. INTRODUCCIÓN

1.3.1 Ventajas del ORM

5. **Lenguaje propio para consultas**
   Las herramientas ORM proporcionan lenguajes específicos para realizar consultas **(HQL en Hibernate, JPQL en JPA)**, sin necesidad de usar SQL directamente.



1. INTRODUCCIÓN

1.3.2 Desventajas del ORM

1. Curva de aprendizaje:

Las herramientas ORM suelen ser complejas y requieren tiempo para aprender a utilizarlas de manera eficiente.

2. Rendimiento:

Las aplicaciones pueden ser más lentas debido al proceso de traducción que realiza el ORM (transformación de consultas, lectura de registros y creación de objetos).

3. Menor control sobre SQL:

La herramienta genera las consultas; si el mapeo está mal configurado, pueden ser ineficientes (lo veremos con la carga perezosa y la ansiosa).



2. Características y herramientas ORM

2.1. Principales características de herramientas ORM

- Las herramientas ORM son soluciones diseñadas para facilitar la interacción entre modelos de datos orientados a objetos y bases de datos relacionales.

- Proporcionan un **enfoque más natural** y eficiente para los desarrolladores (se abstrae de la complejidad del manejo directo de SQL y la gestión de persistencia).

- A continuación, se describen las principales características que definen a las herramientas ORM:



2. Características y herramientas ORM

2.1.1. Técnicas de mapeo

Esenciales en ORM: establecen la correspondencia entre las clases de la aplicación y las tablas de la base de datos.
Existen dos enfoques principales:

- **Definiciones incrustadas en el código**

  Están integradas en las clases de la aplicación mediante mecanismos como las anotaciones en Java y PHP o macros en C++. **Ventaja:** Permite una mayor cohesión entre el modelo de objetos y la configuración de la persistencia.

- **Definiciones en ficheros externos**

  Guardan la configuración en formatos externos como XML o JSON. **Ventaja**: Facilita el mantenimiento y permite modificar las definiciones sin alterar el código fuente.

Ambos métodos pueden coexistir en una misma aplicación, ofreciendo flexibilidad al programador. En esta unidad veremos los dos: ficheros **.hbm.xml** y **anotaciones JPA**.



2. Características y herramientas ORM

2.1.2. Lenguaje de consulta orientado a objetos (OQL)

Cada herramienta ORM incorpora su propio lenguaje de consultas, basado en objetos en lugar de tablas: HQL en Hibernate y JPQL en JPA.

Diferencias con SQL:

- **Acceso orientado a objetos:**

  Las consultas trabajan directamente con objetos, no con filas o columnas.

- **Traducción automática**

  Traducen estas consultas a SQL en segundo plano.
  Por ejemplo:

```
SELECT a FROM Album a WHERE a.titulo = 'Black'
```

Esta consulta JPQL recupera los objetos tipo Álbum cuyo título sea "Black", sin que el programador deba escribir una consulta SQL explícita.



2. Características y herramientas ORM

2.1.3. Técnicas de sincronización

- Una de las funciones más complejas e importantes de las herramientas ORM.
- Se encarga de garantizar que los cambios en los objetos se reflejen en la base de datos y viceversa.

1. **Detección de cambios en los objetos:**

   La herramienta supervisa las modificaciones realizadas en los objetos durante su ciclo de vida para sincronizarlas con la base de datos (**en JPA se llama dirty checking**).





2. Características y herramientas ORM

2.1.3. Técnicas de sincronización



2. Creación de instancias de objetos:

A partir de los datos almacenados en la base de datos, el ORM genera instancias de objetos completos en el lenguaje de programación.

3. Persistencia automática:



Con una simple instrucción como em.persist(objeto), se generan automáticamente las sentencias SQL necesarias.

```
em.persist(album1);
```

→ Inserta los datos de album1 en las tablas correspondientes



2. Características y herramientas ORM

2.1.3. Técnicas de sincronización

4. Consultas transparentes:

Convierten consultas complejas en un proceso sencillo y manejable mediante su lenguaje de consultas (JPQL o HQL), devolviendo resultados en forma de objetos que el programador puede usar directamente.





2. Características y herramientas ORM









Gracias a estas características, las herramientas ORM reducen la necesidad de que los desarrolladores gestionen directamente sentencias SQL complejas o realicen conversiones manuales entre objetos y datos relacionales.

- **Mayor productividad**: los desarrolladores se centran en la lógica de negocio en lugar de en la gestión de la persistencia.
- **Reducción de errores:** al automatizar la generación de consultas y la sincronización de datos, se minimizan los errores típicos de manipulación manual.
- **Código más limpio**: la aplicación no incluye código repetitivo para convertir filas en objetos y viceversa.

2. Características y herramientas ORM

2.2. Herramientas ORM más utilizadas en el mercado

Existen diversas herramientas ORM, diseñadas para diferentes lenguajes de programación. Algunas de las más destacadas son:

¿?



2. Características y herramientas ORM

2.2. Herramientas ORM más utilizadas en el mercado

Lenguaje	ORM	Descripción
Java	**Hibernate**	Permite mapear prácticamente la totalidad de los SGBDR, como MySQL, Postgres, Oracle y MS SQL Server.
	EclipseLink	Implementación de referencia de JPA; alternativa a Hibernate.
	Morphia	Mapeo para la base de datos MongoDB **(ODM: mapeo objeto-documento)**.
Node.js	**Sequelize**	Mapeo para los SGBDR, prácticamente los mismos que Hibernate.
	Mongoose	Mapeo para MongoDB **(ODM: mapeo objeto-documento)**.
Python	**SQLAlchemy**	Permite acceso a las bases de datos relacionales más habituales.
	SQLObject	
	Tortoise ORM	
	MongoEngine	Permite gestionar el acceso a MongoDB **(ODM: mapeo objeto-documento)**.
	Beanie	
C# / .NET	**Entity Framework Core**	ORM oficial de Microsoft.
PHP	**Doctrine / Eloquent**	ORM de Symfony y Laravel.



Evolución de las herramientas

Estas herramientas han evolucionado para ofrecer soporte para múltiples bases de datos y proporcionar lenguajes propios de consulta que simplifican la interacción con las bases de datos.



2. Características y herramientas ORM

2.3. Hibernate: ¿Qué es?

- **Framework ORM:** Hibernate es un framework de mapeo objeto-relacional (ORM) diseñado para aplicaciones Java.
- **Objetivo principal:** facilitar el mapeo de atributos entre el modelo de objetos de una aplicación y una base de datos relacional, permitiendo que el desarrollador trabaje directamente con objetos mientras Hibernate gestiona las interacciones con la base de datos.
- **Configuración:** utiliza ficheros XML o anotaciones en las propias clases Java.
- **Licencia:** Hibernate es software libre. Desde la versión 7 se distribuye con licencia Apache 2.0 (antes, LGPL 2.1). Se puede utilizar en aplicaciones comerciales sin la obligación de hacer público el código fuente del proyecto.
- **Adopción:** gracias a esta flexibilidad, Hibernate se ha consolidado como una herramienta ampliamente adoptada en el desarrollo empresarial.



2. Características y herramientas ORM

2.3. Hibernate: Origen y evolución

- Hibernate fue desarrollado en 2001 como una alternativa a la persistencia de objetos proporcionada en Java EE mediante los Enterprise Java Beans (EJB). Aunque Java EE es una plataforma potente, también es compleja y exige muchos recursos. Hibernate surgió como una solución más ligera y sencilla, ofreciendo persistencia de objetos sin la necesidad de un servidor de aplicaciones empresarial.

- **En 2010 (versión 3.5) se convirtió en implementación certificada de JPA. Su versión actual, la 7, implementa Jakarta Persistence 3.2**.



2. Características y herramientas ORM

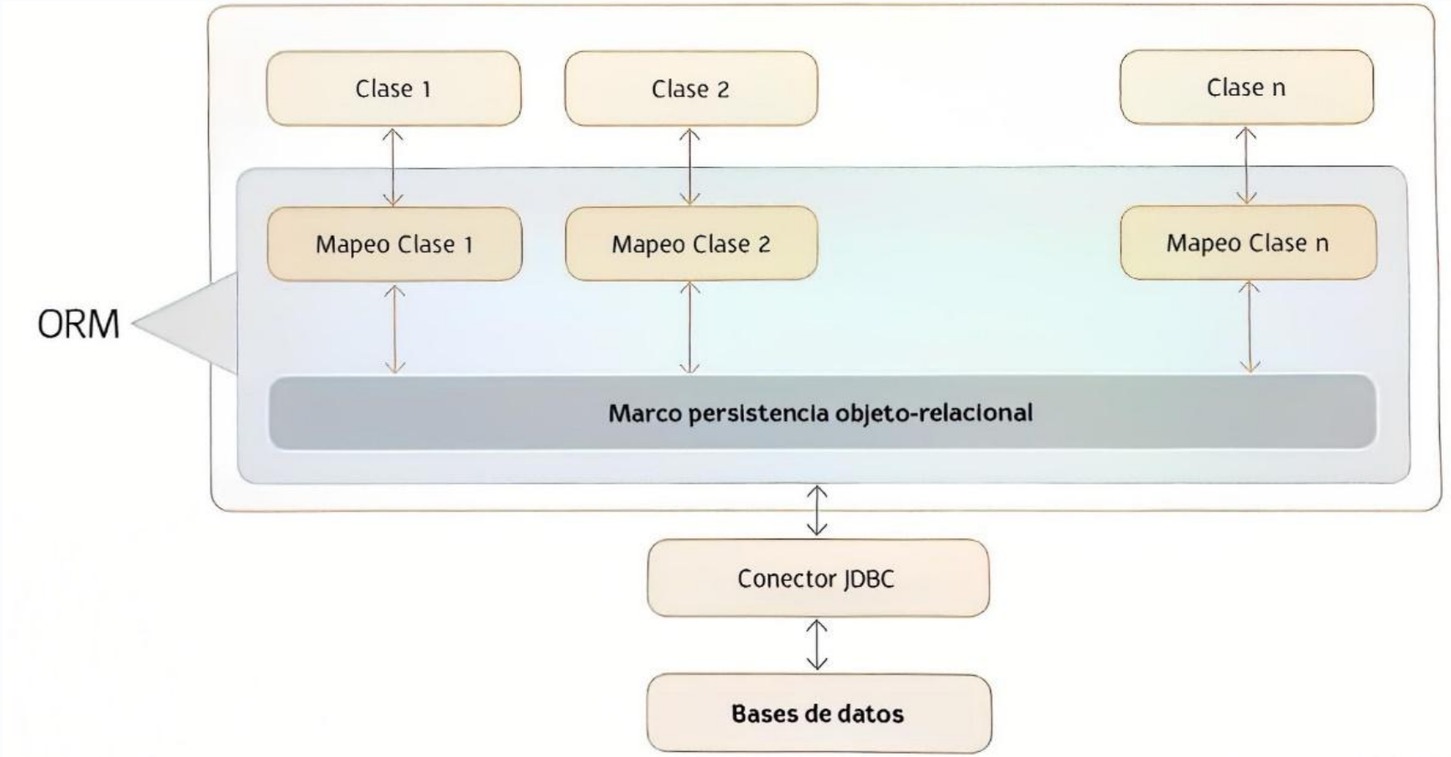
2.3. Hibernate: HQL y portabilidad

- Hibernate incluye un **lenguaje de consultas denominado HQL**, basado en objetos y similar a SQL.
- A diferencia de SQL, **HQL opera sobre clases y atributos** en lugar de tablas y columnas.
- Esto permite a los desarrolladores formular **consultas de manera más intuitiva**, utilizando las propiedades de los objetos.
- Hibernate traduce estas consultas al lenguaje SQL específico del motor de base de datos utilizado, **facilitando la portabilidad entre distintos sistemas gestores de bases de datos (SGBD).**



2. Características y herramientas ORM

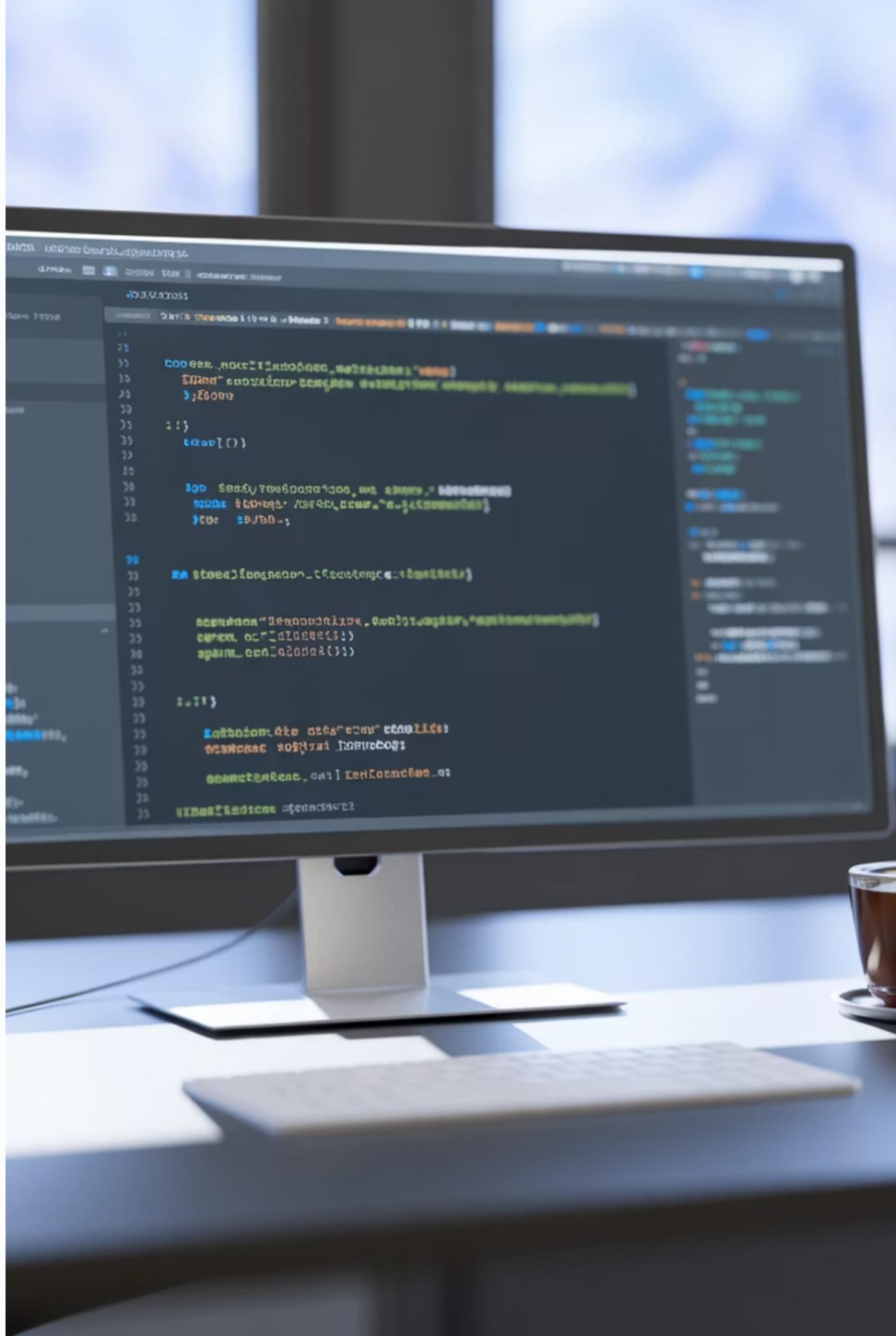
2.3. Hibernate: HQL y portabilidad



2. Características y herramientas ORM

2.4. JPA e Hibernate

La especificación JPA y su implementación con Hibernate



2.4. JPA e Hibernate

La alternativa estándar

¿Qué es JPA (Java Persistence API)?

- JPA **no es una herramienta**, es una **especificación** (un "manual de instrucciones" o un estándar).
- Define un conjunto de reglas e interfaces (como EntityManager) sobre cómo debe funcionar la persistencia en Java.
- **Analogía:** JPA es el estándar "USB-C".

¿Qué es Hibernate entonces?

- Hibernate es una **implementación** de JPA. Es la *herramienta* (el "motor") que sabe leer ese manual y hacer el trabajo.
- **Analogía:** Hibernate es el "cargador de Samsung" que *usa* el estándar USB-C.

¿Por qué usar JPA?

- **Portabilidad:** Tu código (tu main) usará clases estándar de jakarta.persistence.*. Si un día quieres cambiar el "motor" de Hibernate a otro (como EclipseLink), tu código apenas cambia.
- Desde 2019, JPA se llama Jakarta Persistence. Por eso los paquetes son **jakarta.persistence.*** (en tutoriales antiguos verás javax.persistence.*).

JPA vs Hibernate

Característica	API de JPA	API Nativa de Hibernate
Estándar	Sí, es el estándar de Java (JPA).	No, es específico de Hibernate.
Fichero Config.	persistence.xml	hibernate.cfg.xml
Clase Principal	jakarta.persistence.Persistence	org.hibernate.cfg.Configuration
La "Fábrica"	EntityManagerFactory	SessionFactory
La "Sesión"	EntityManager	Session

3. Instalación y configuración de una herramienta ORM

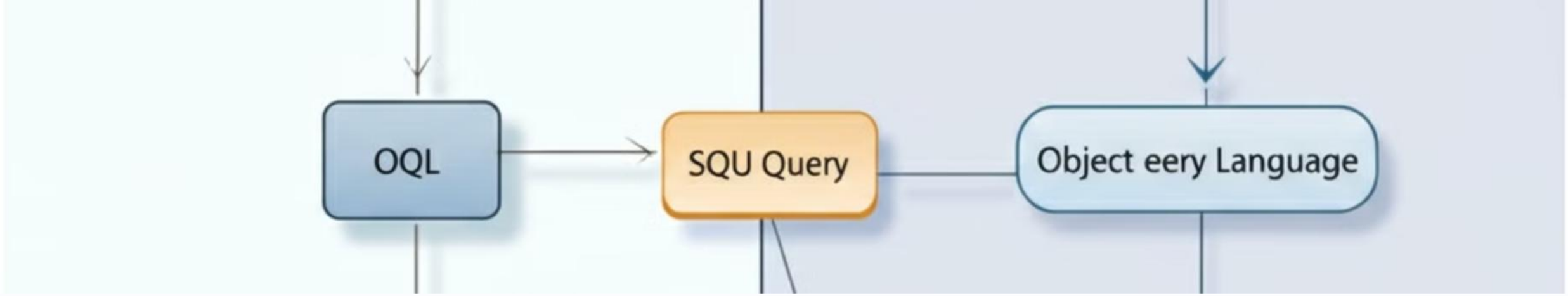
3.1. Instalación de la herramienta ORM (Hibernate)

3.2. Configuración inicial

Estos apartados de la unidad de trabajo 3, se guían y desarrollas a través de:

'Actividad 1. Instalación y configuración de Hibernate'.





3.2. Configuración inicial de Hibernate

- **Configuración inicial:** Ficheros de configuración y propiedades configurables
- Hibernate requiere la configuración de varios ficheros clave para su correcto funcionamiento
- Estos ficheros proporcionan información necesaria para:
  - Establecer la conexión con la base de datos
  - Gestionar la persistencia
  - Realizar tareas avanzadas como la ingeniería inversa

3. Instalación y configuración de una herramienta ORM

FICHEROS



FICHEROS DE MAPEO O CORRESPONDENCIA

.hbm.xml

Especifican la correspondencia entre clases de Java y tablas de una base de datos relacional.



POJO

.java

Representa la entidad en el código Java. Cada instancia de la clase POJO es un objeto en la aplicación.



3. Instalación y configuración de una herramienta ORM

FICHEROS NECESARIOS PARA CREAR EL MAPEO



CONFIGURACIÓN

hibernate.cfg.xml

Fichero que contiene lo necesario para establecer una conexión con la BBDD

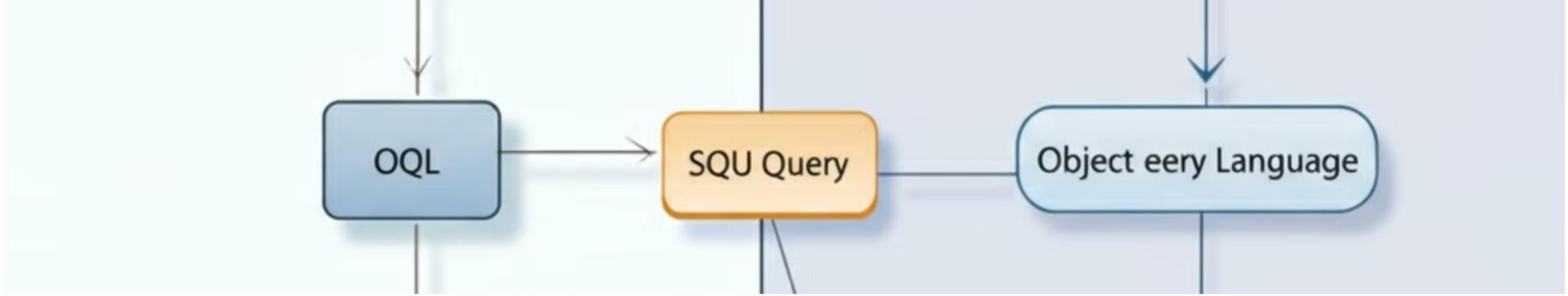


INGENIERÍA INVERSA

hibernate.reveng.xml

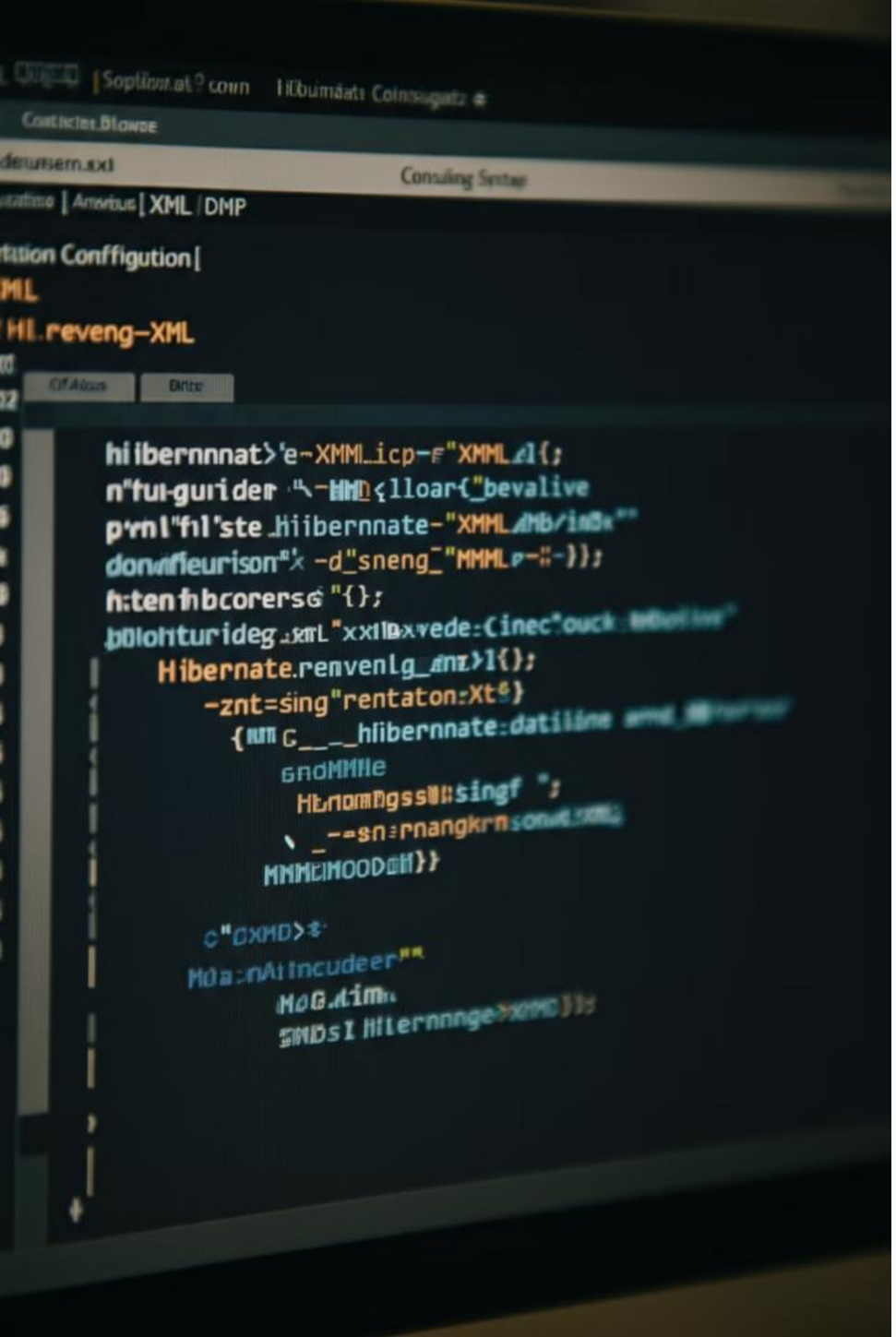
Especifica para qué clases y tablas se va a establecer la correspondencia





3.2.1. Fichero de configuración principal: hibernate.cfg.xml

- El archivo hibernate.cfg.xml es el núcleo de la configuración de Hibernate.
- Contiene la información básica sobre la conexión a la base de datos y las propiedades necesarias para que Hibernate funcione.
- Este archivo suele incluir una serie de parámetros.

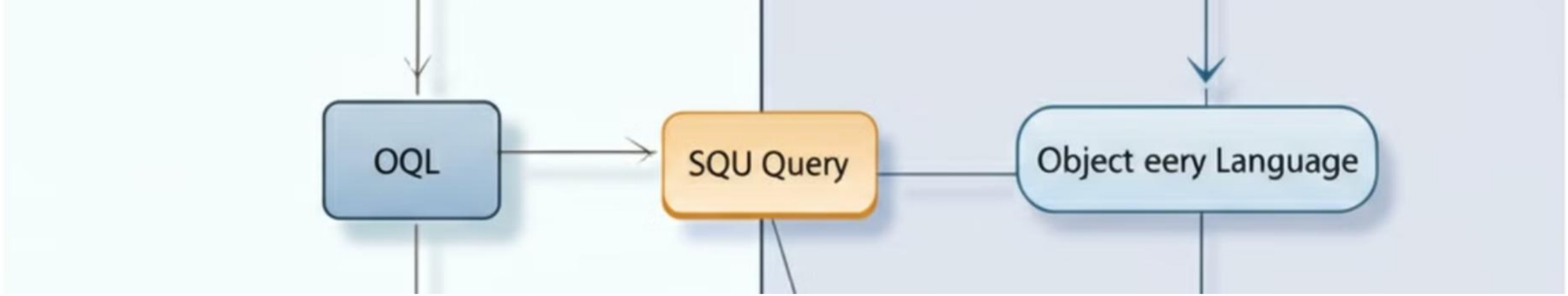


Propiedades principales de hibernate.cfg.xml:

- **hibernate.dialect:** Define el dialecto específico de la base de datos utilizada. Por ejemplo, para MySQL se utiliza org.hibernate.dialect.MySQLDialect.
- **hibernate.connection.driver_class:** Indica el driver JDBC necesario para la conexión (por ejemplo, com.mysql.cj.jdbc.Driver).
- **hibernate.connection.url:** URL de la base de datos a la que se conectará Hibernate.
- **hibernate.connection.username:** Nombre de usuario para acceder a la base de datos (por defecto, suele ser root en MySQL).
- **hibernate.connection.password:** Contraseña asociada al usuario.
- **hibernate.show_sql:** Permite mostrar las sentencias SQL generadas por Hibernate en la consola (valor true o false).

Ejemplo de hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory name="SesionLoren">
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.password">loren</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/loren</property>
        <property name="hibernate.connection.username">loren</property>
        <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    </session-factory>
</hibernate-configuration>
```



3.2.2. Fichero de ingeniería inversa: hibernate.reveng.xml

- El archivo **hibernate.reveng.xml** se utiliza para definir el esquema y las tablas de la base de datos que se desea mapear automáticamente.
- Es una herramienta útil para generar las clases Java correspondientes a las tablas existentes en una base de datos.

Ejemplo de hibernate.reveng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-reverse-engineering PUBLIC "-//Hibernate/Hibernate Reverse Engineering DTD
3.0//EN" "http://hibernate.org/dtd/hibernate-reverse-engineering-3.0.dtd" >

<hibernate-reverse-engineering>
  <table-filter match-catalog="loren" match-name="agentes"/>
  <table-filter match-catalog="loren" match-name="clientes"/>
  <table-filter match-catalog="loren" match-name="datos_climaticos"/>
  <table-filter match-catalog="loren" match-name="llamadas_emitidas"/>
  <table-filter match-catalog="loren" match-name="tarjetas_telefonicas"/>
</hibernate-reverse-engineering>
```

Comparativa de ficheros: Hibernate nativo vs. JPA

Hibernate NATIVO

01

hibernate.cfg.xml

Contiene la conexión y el dialecto.

02

hibernate.reveng.xml

Define qué tablas mapear.

03

MiClase.java (POJO)

La clase Java vacía.

04

MiClase.hbm.xml

El "puente" que une el POJO con la tabla.

Estándar JPA

01

persistence.xml

Contiene la conexión, el dialecto y define la "Unidad de Persistencia". **Sustituye a hibernate.cfg.xml**.

02

MiClase.java (POJO + Anotaciones)

Sustituye al .hbm.xml.

El "mapeo" se escribe directamente en la clase Java usando anotaciones (ej. @Entity, @Table, @Id).

¡No se necesita un fichero .hbm.xml!

Fichero de configuración estándar: persistence.xml



Es el "núcleo" de la configuración de JPA. **Sustituye a hibernate.cfg.xml**.

Se ubica siempre en una carpeta llamada META-INF (dentro de resources).

Define una o más **"Unidades de Persistencia"** (<persistence-unit>).

- Una "Unidad de Persistencia" es un conjunto de clases (Entidades) que se gestionarán juntas.

¡Importante! Dentro de este fichero estándar, seguimos teniendo que decirle a JPA qué "motor" (provider) vamos a usar (en nuestro caso, Hibernate).

Comparativa de propiedades: cfg.xml vs persistence.xml

En hibernate.cfg.xml (Nativo)

- <property name="hibernate.dialect">...
- <property name="hibernate.connection.driver_class">...
- <property name="hibernate.connection.url">...
- <property name="hibernate.connection.username">...
- <property name="hibernate.connection.password">...

En persistence.xml (estándar JPA)

- <property name="jakarta.persistence.jdbc.driver" ... />
- <property name="jakarta.persistence.jdbc.url" ... />
- <property name="jakarta.persistence.jdbc.user" ... />
- <property name="jakarta.persistence.jdbc.password" ... />
- <property name="hibernate.dialect" ... />



¡OJO! El dialecto es una propiedad *específica* de Hibernate, ¡pero el estándar JPA nos deja incluirla!

Ejemplo de persistence.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd"
         version="3.2">
   <persistence-unit name="default">
      <properties>
         <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
         <property name="jakarta.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/ut3"/>
         <property name="jakarta.persistence.jdbc.user" value="root"/>
         <property name="jakarta.persistence.jdbc.password" value="123456"/>
         <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect"/>
      </properties>
   </persistence-unit>
</persistence>
```

El mapeo moderno: anotaciones (JPA)

(Adiós, .hbm.xml!)



@Entity

Le dice a JPA: "Esta clase es una tabla".



@Table(name=...)

(Opcional) Especifica el nombre real de la tabla si es diferente al de la clase.



@Id

Marca la propiedad que es la clave primaria.



@Column(name=...)

(Opcional) Especifica el nombre real de la columna.

Ejemplo de POJO (LlamadasEmitida.java) con Anotaciones

En JPA, el mapeo se fusiona con el POJO.

Usamos "Anotaciones" (etiquetas que empiezan por @) para describir el mapeo.

La "Ingeniería Inversa" moderna (como la opción Create JPA Entities from DB...) genera **automáticamente** estas clases con sus anotaciones.

Ejemplo de POJO (LlamadasEmitida.java) sin anotaciones (Hibernate)

```java
package org.example;

/**
 * LlamadasEmitidas generated by hbm2java
 */
public class LlamadasEmitida implements java.io.Serializable {
    private int codigoLlamada;
    private int numeroLlamado;
    private int duracionLlamada;
    private float importeLlamada;
    public LlamadasEmitida() {
    }
    public LlamadasEmitida(int codigoLlamada, int numeroLlamado,
                           int duracionLlamada, float importeLlamada) {
        this.codigoLlamada = codigoLlamada;
        this.numeroLlamado = numeroLlamado;
        this.duracionLlamada = duracionLlamada;
        this.importeLlamada = importeLlamada;
    }
    public int getCodigoLlamada() {
        return this.codigoLlamada;
    }
    public void setCodigoLlamada(int codigoLlamada) {
        this.codigoLlamada = codigoLlamada;
    }
    public int getNumeroLlamado() {
        return this.numeroLlamado;
    }
    public void setNumeroLlamado(int numeroLlamado) {
        this.numeroLlamado = numeroLlamado;
    }
    public int getDuracionLlamada() {
        return this.duracionLlamada;
    }
    public void setDuracionLlamada(int duracionLlamada) {
        this.duracionLlamada = duracionLlamada;
    }
    public float getImporteLlamada() {
        return this.importeLlamada;
    }
    public void setImporteLlamada(float importeLlamada) {
        this.importeLlamada = importeLlamada;
    }
}
```

Ejemplo de POJO (LlamadasEmitida.java) con anotaciones (JPA)

```java
package org.example;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "LLAMADAS_EMITIDAS")
public class LlamadasEmitida {
    @Id
    @Column(name = "CODIGO_LLAMADA", nullable = false)
    private Integer id;

    @Column(name = "NUMERO_LLAMADO", nullable = false)
    private Integer numeroLlamado;

    @Column(name = "DURACION_LLAMADA", nullable = false)
    private Integer duracionLlamada;

    @Column(name = "IMPORTE_LLAMADA", nullable = false)
    private Float importeLlamada;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public Integer getNumeroLlamado() {
        return numeroLlamado;
    }

    public void setNumeroLlamado(Integer numeroLlamado) {
        this.numeroLlamado = numeroLlamado;
    }

    public Integer getDuracionLlamada() {
        return duracionLlamada;
    }

    public void setDuracionLlamada(Integer duracionLlamada) {
        this.duracionLlamada = duracionLlamada;
    }

    public Float getImporteLlamada() {
        return importeLlamada;
    }

    public void setImporteLlamada(Float importeLlamada) {
        this.importeLlamada = importeLlamada;
    }
}
```

- En JPA, el mapeo se fusiona con el POJO.
- Usamos "Anotaciones" (etiquetas que empiezan por @) para describir el mapeo.
- La "Ingeniería Inversa" moderna (como la opción Create JPA Entities from DB... En IntelliJ) genera **automáticamente** estas clases con sus anotaciones.

4. Ficheros de mapeo o correspondencia

Estos archivos contienen información clave sobre la correspondencia entre diferentes sistemas y formatos de datos.



4.2. Estructura de un fichero de mapeo

Definición

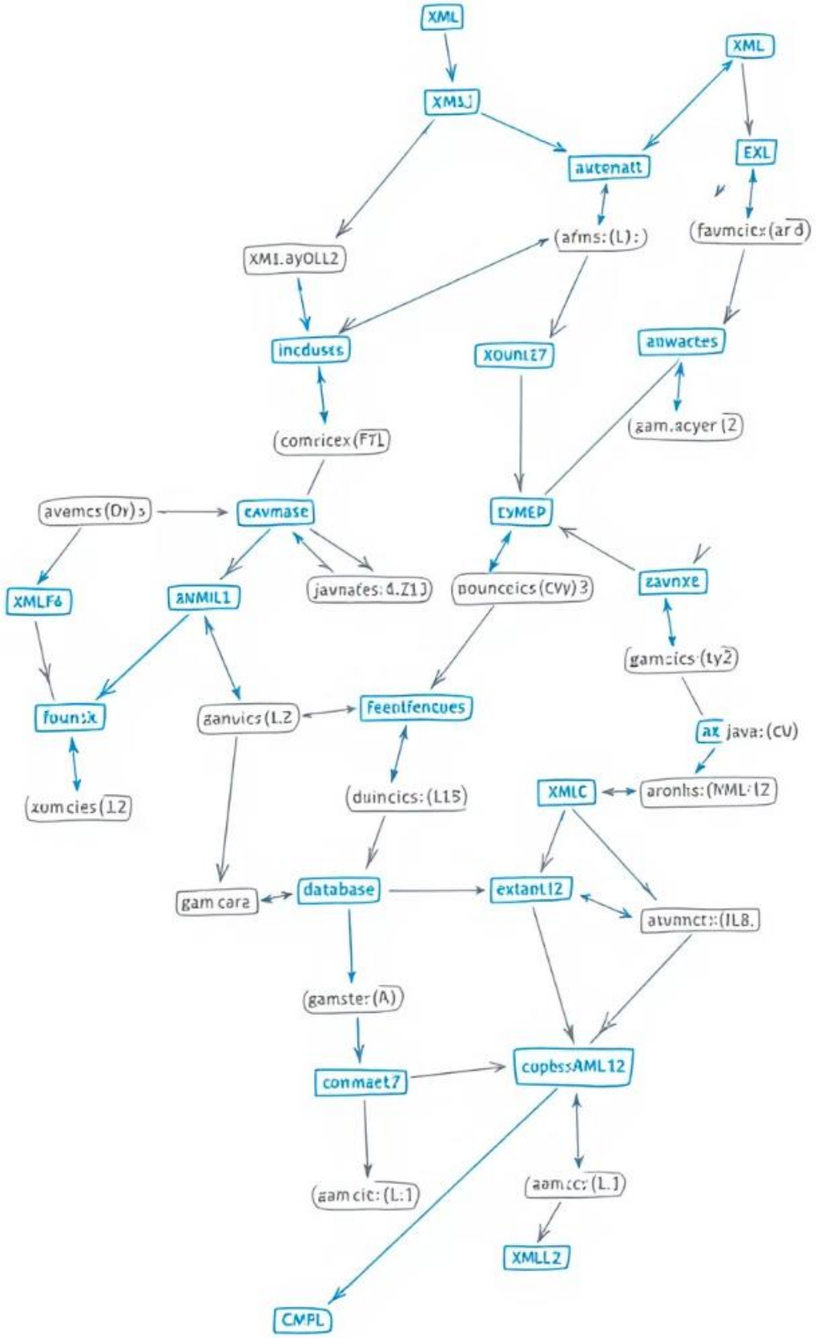
Un fichero de mapeo **.hbm.xml** define la correspondencia entre una clase Java y su tabla relacional asociada.

Contenido

Contiene las reglas necesarias para convertir los objetos Java en datos persistentes y viceversa.

Ubicación

Este archivo es obligatorio para cada clase que queremos hacer persistente y suele colocarse en el directorio resources, o en el mismo paquete que la clase correspondiente.



Elementos de .hbm.xml

hibernate-mapping

Indica que este archivo contiene definiciones de mapeo de Hibernate.

class

Representa la clase Java y su tabla correspondiente, con atributos como **name**, **table** y **schema**.

id

Define el atributo que actúa como clave primaria, especificando cómo se generarán los valores del ID.

property

Mapea los atributos de la clase con las columnas de la tabla.

Ejemplo de estructura básica de un fichero de mapeo

```xml
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

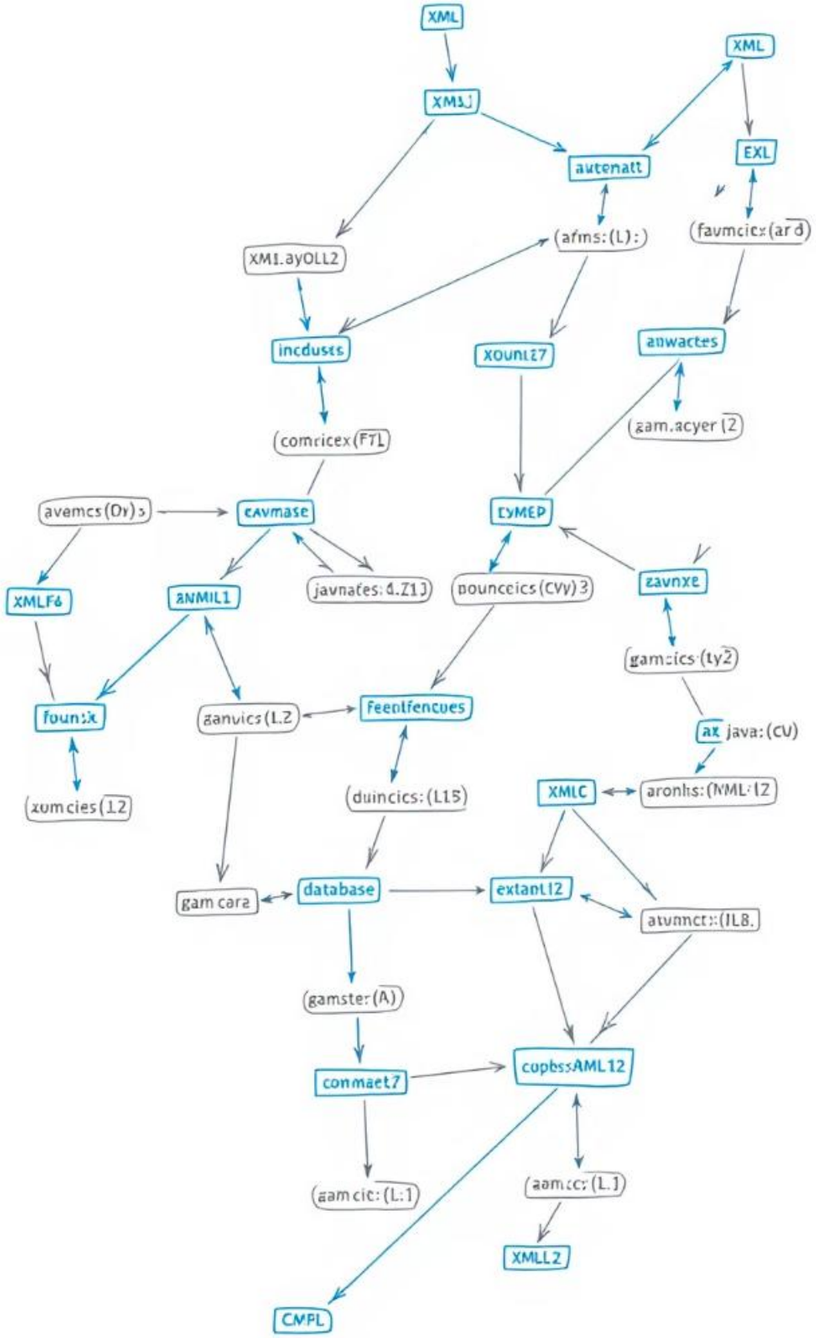
<hibernate-mapping>
   <class name = "Employee" table = "EMPLOYEE">

      <meta attribute = "class-description">
         This class contains the employee detail.
      </meta>

      <id name = "id" type = "int" column = "id">
         <generator class="native"/>
      </id>

      <property name = "firstName" column = "first_name" type = "string"/>
      <property name = "lastName" column = "last_name" type = "string"/>
      <property name = "salary" column = "salary" type = "int"/>

   </class>
</hibernate-mapping>
```

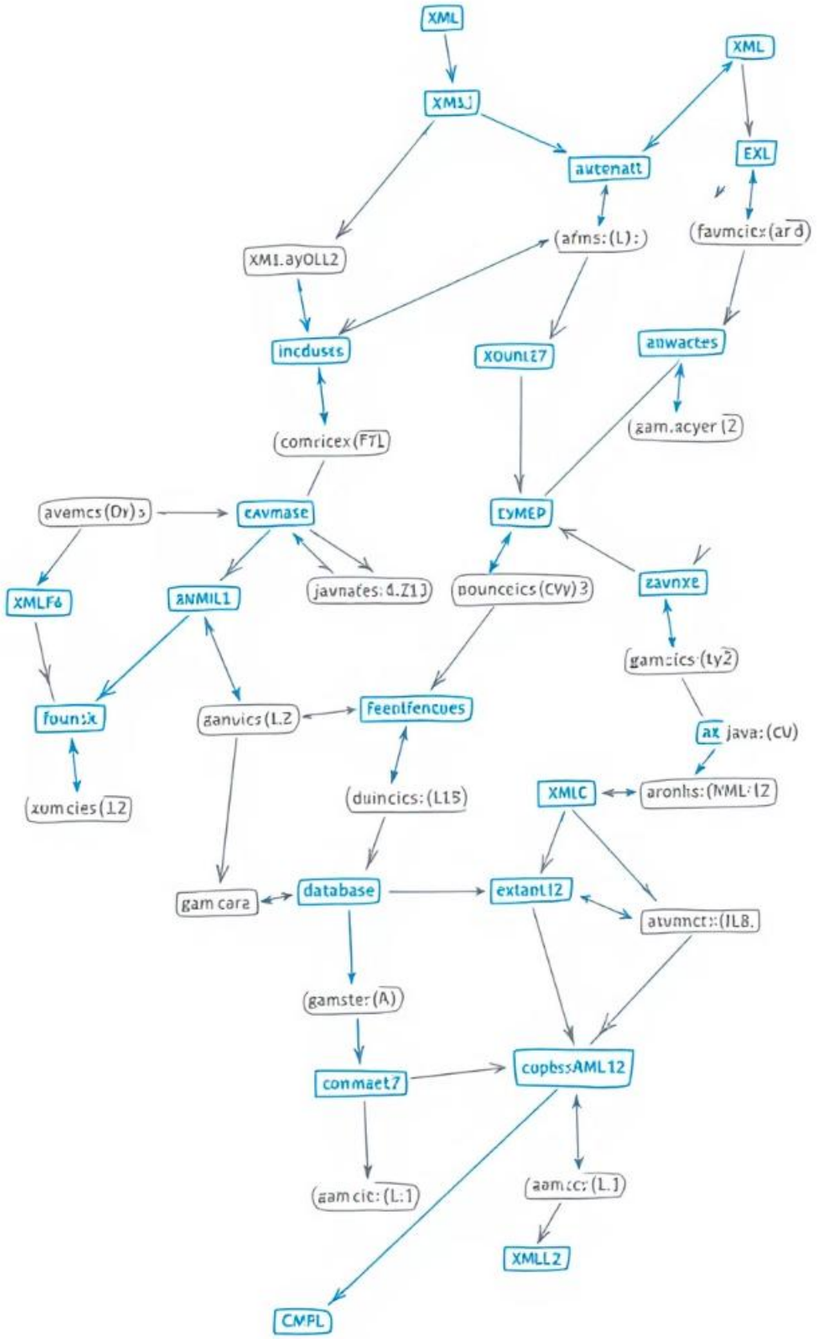


Revisión

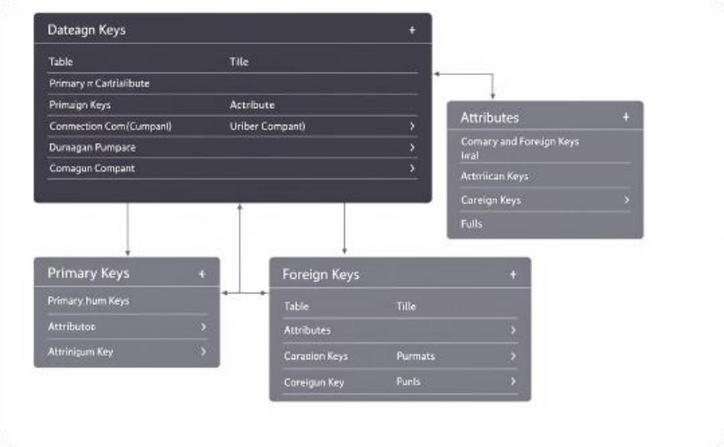
En relación a Hibernate

- ¿Qué es el fichero de configuración? (qué contiene, y para qué se utiliza).
- ¿Qué es el fichero de ingenieria inversa? (qué contiene y para qué se utiliza)
- ¿Qué es el fichero de mapeo? (qué contiene y para qué se utiliza).
- ¿Que es un POJO?





4.3. Elementos y propiedades mapeables



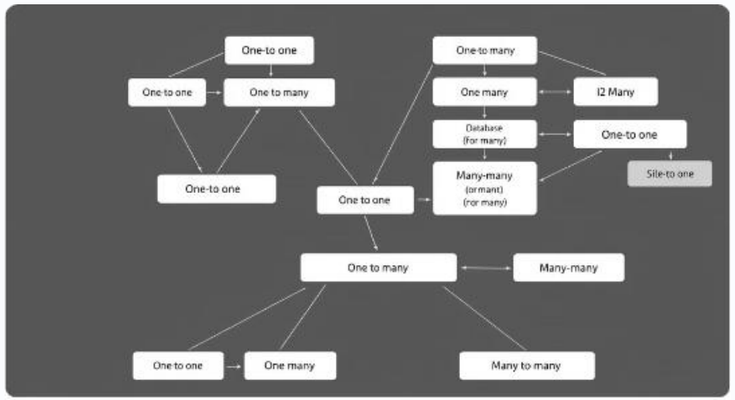
Elementos mapeables

Hibernate permite mapear diversos elementos entre tablas y clases, adaptándose a múltiples necesidades.

Nates	Sample Data	Renprisn	Dirk Table
12/141/2016, (Getan)	2: U.51, Fetsh	Tisk 1	Tisk_:_Netro:
12/141/2015, (Getan)	2: U.34, Compiny Data)	Disk 4	Tisk_:_Netro:
12/141/2015, (Getan)	3: U.91; Latck	Disk 7	Tisk_:_New Rectors Daba)
12/141/2016, (Getan)	3: U.94, Cample Fetra)	Dist 5	Tisk_:_Netro:
12/141/2018, (Getan)	3: U.51,_Cence)	Dist 4	Tisk_:_New Reciure Data)
12/141/2018, (Getan)	3: U.91, Carue)	Dist 4	Tisk_:_Netro:
12/141/2015, (Getan)	3: U.57, CallerFetra)	Dark 7	Tisk_:_Manule Ducs Taba)
12/145/2019, (Getan)	3: U.92, Carne)	Tixt 4	Tisk_:_Netro:
12/141/2018, (Getan)	6: U.66, CallerFetra)	Dist 5	Tisk_:_Netro:
12/141/2018, (Getan)	5: U.54,_Crespry	Dist 4	Tisk_:_Netro:
12/141/2014, (Getan)	8: U.94, Cample Data)	Dost 11	Tisk_:_New Reciure Data)
12/141/2019, (Getan)	6: U.54, CallerFerra)	Tist 5	Tisk_:_Netro:
12/141/2018, (Getun)	15/0:15 **Sample Data**		

Propiedades mapeables

Las **propiedades** de las clases se pueden mapear a columnas de las tablas de la base de datos.



Relaciones mapeables

Relaciones entre tablas, como one-to-many o many-to-many, también pueden ser mapeadas.

4.3. Elementos y propiedades mapeables

Ejemplo práctico: ALBUM y CANCIONES





Elementos mapeables

Los elementos son las entidades o clases principales que representan las tablas de la base de datos:

- Clase álbum.
- Clase canción

Propiedades mapeables

Son los atributos dentro de las clases que se corresponden con columnas en las tablas de la base de datos.

- Id, título y autor (de álbum)
- Id, título y duración (de canción)

Relaciones mapeables

Establecen vínculos entre las entidades y se corresponden con las relaciones entre tablas en la base de datos.

- Un álbum y sus canciones (One to many).
- Una canción y el álbum al que pertenece (Many to one)...

Relación POJOs y ficheros de mapeo

1 Definición de POJOs

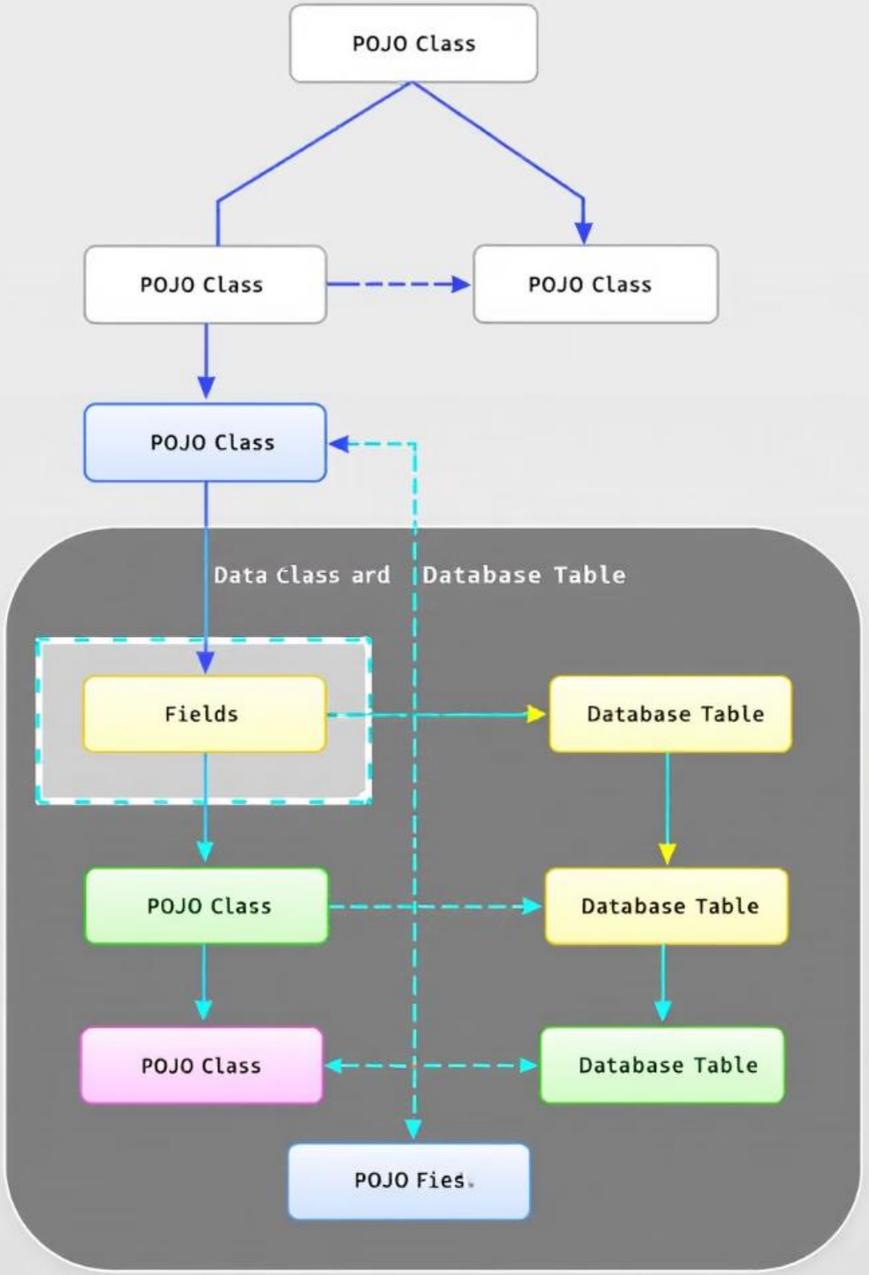
Los POJOs son clases Java simples que representan en Java entidades de la base de datos.

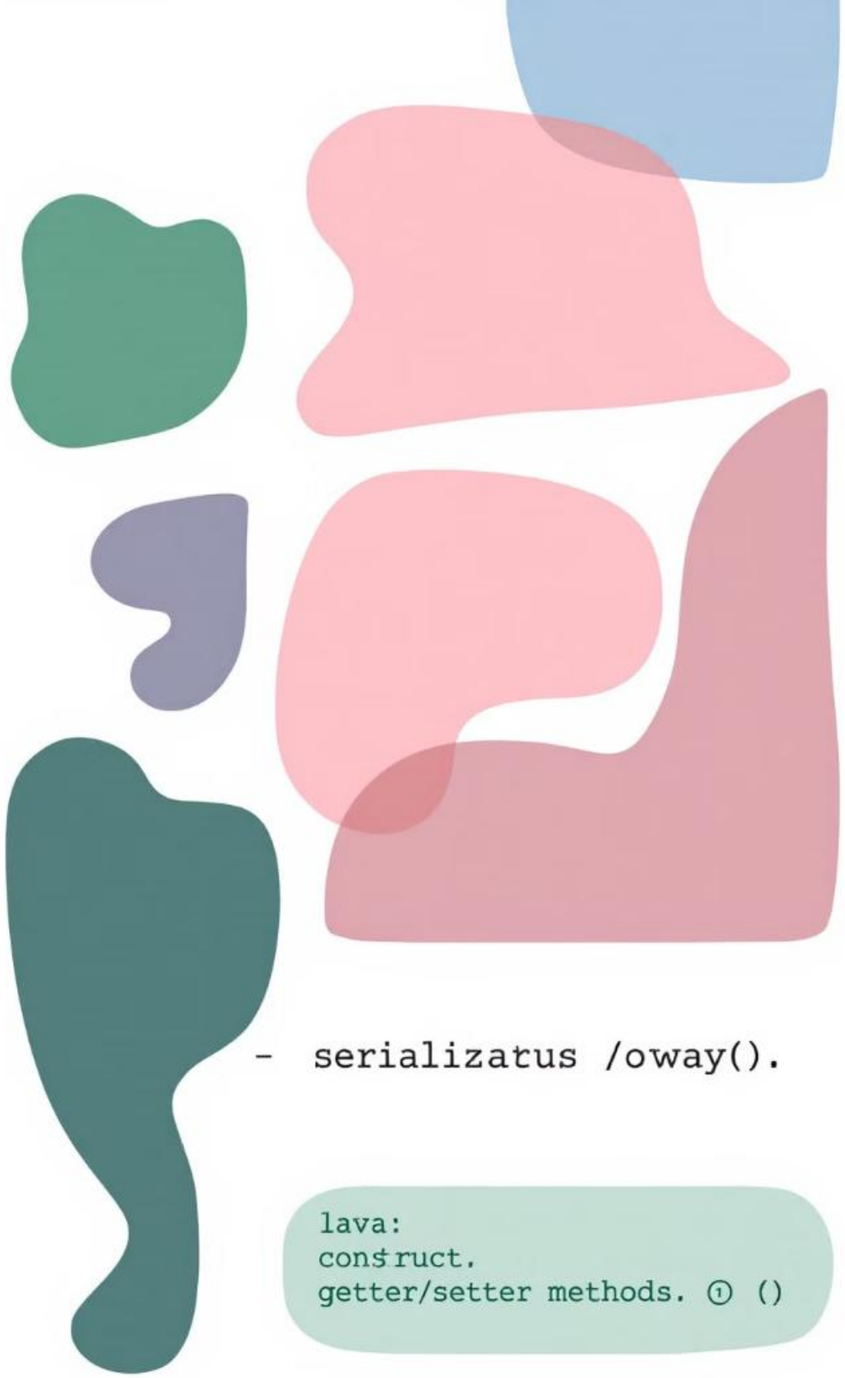
2 Ficheros de mapeo

Los ficheros de mapeo (.hbm.xml) relacionan los POJOs con las tablas de la base de datos.

3 Asignación de datos

Los ficheros de mapeo definen cómo se asignan los datos de las columnas a los atributos de los POJOs.





Características de las clases Java (POJO)

Estructura básica (para que sean clases persistentes)

- Deben implementar la interfaz **java.io.Serializable**.
- Contar con un constructor público sin argumentos.
- Utilizar atributos privados (métodos getter y setter).

Reglas de mapeo

- Los nombres de los atributos de las clases se derivan de los nombres de las columnas en las tablas.

Ejemplo de POJO

Un **POJO (Plain Old Java Object)** es una clase Java sencilla que no hereda de ninguna clase específica ni implementa interfaces especiales.

Los POJOs se caracterizan por tener **propiedades privadas** y **métodos de acceso (getters y setters)** para manipular esos datos.

```java
public class Album implements java.io.Serializable {
    private Long id;
    private String titulo;
    private String autor;

    public Album() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

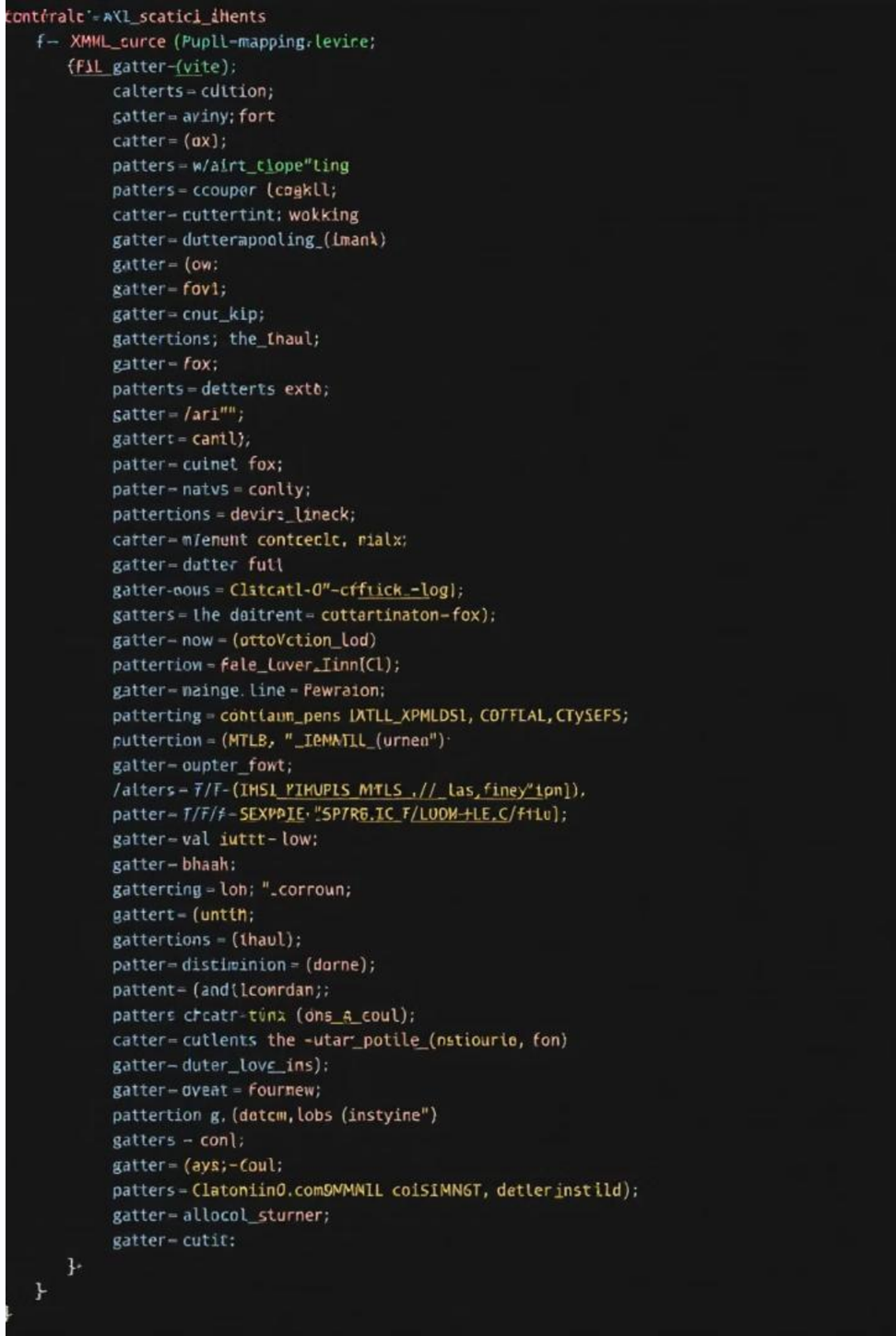
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }

    public String getAutor() { return autor; }
    public void setAutor(String autor) { this.autor = autor; }
}
```

Ejemplo de fichero de mapeo

El fichero de mapeo **hbm.xml** es un archivo XML que define la relación entre las clases Java (POJOs) y las tablas de la base de datos.

```xml
<hibernate-mapping>
    <class name="com.example.Album" table="ALBUMES">
        <id name="id" column="ID" type="long">
            <generator class="identity"/>
        </id>
        <property name="titulo" column="TITULO" type="string"/>
        <property name="autor" column="AUTOR" type="string"/>
    </class>
</hibernate-mapping>
```



Relación práctica entre un POJO y un fichero de mapeo

1

POJO (datos en memoria)

Cada instancia de la clase POJO representa los datos en la aplicación, como el título y autor de un álbum.

2

Fichero de Mapeo

Define cómo los atributos del POJO se almacenan en las columnas de la tabla, permitiendo a Hibernate gestionar la persistencia.

3

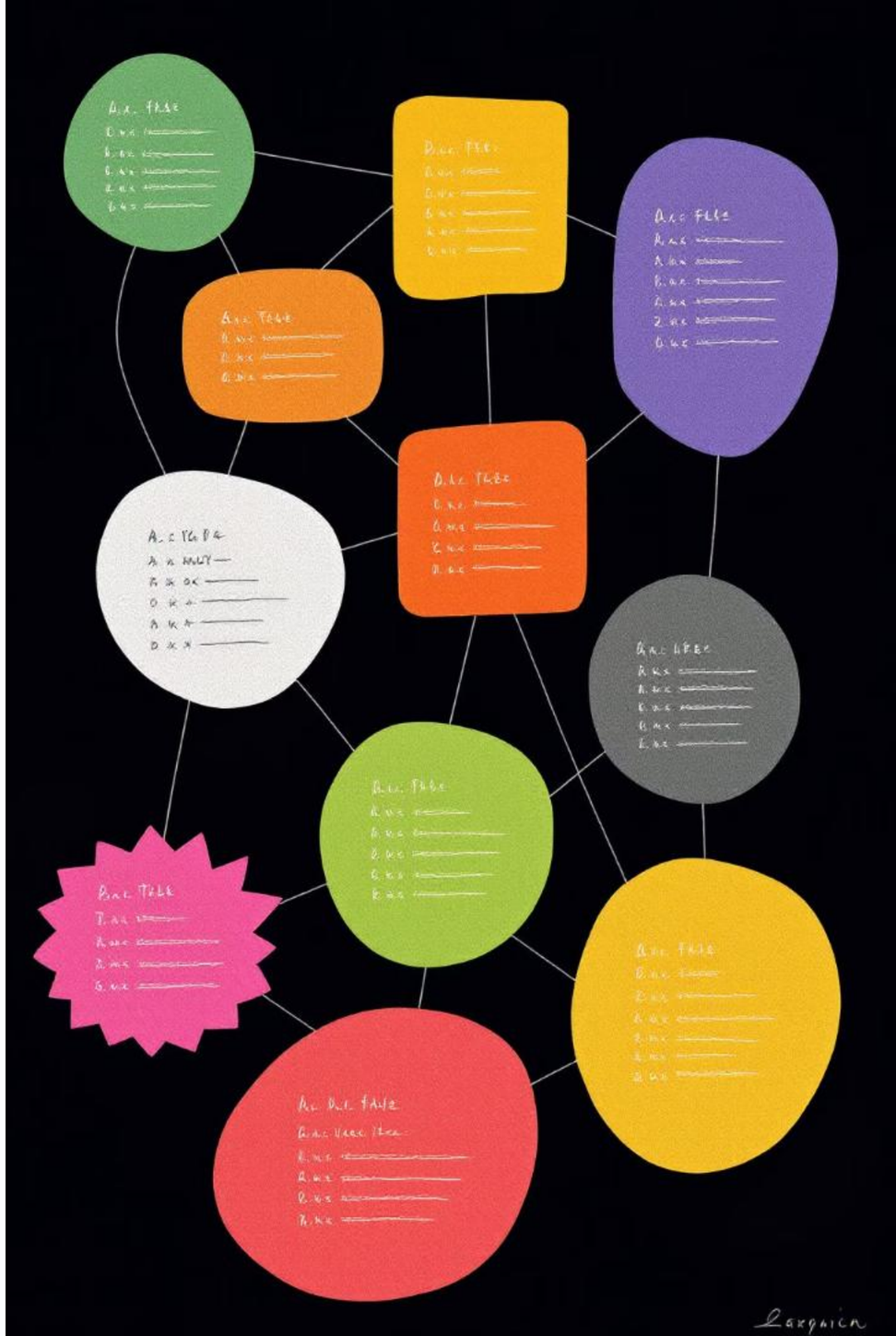
Operaciones de persistencia

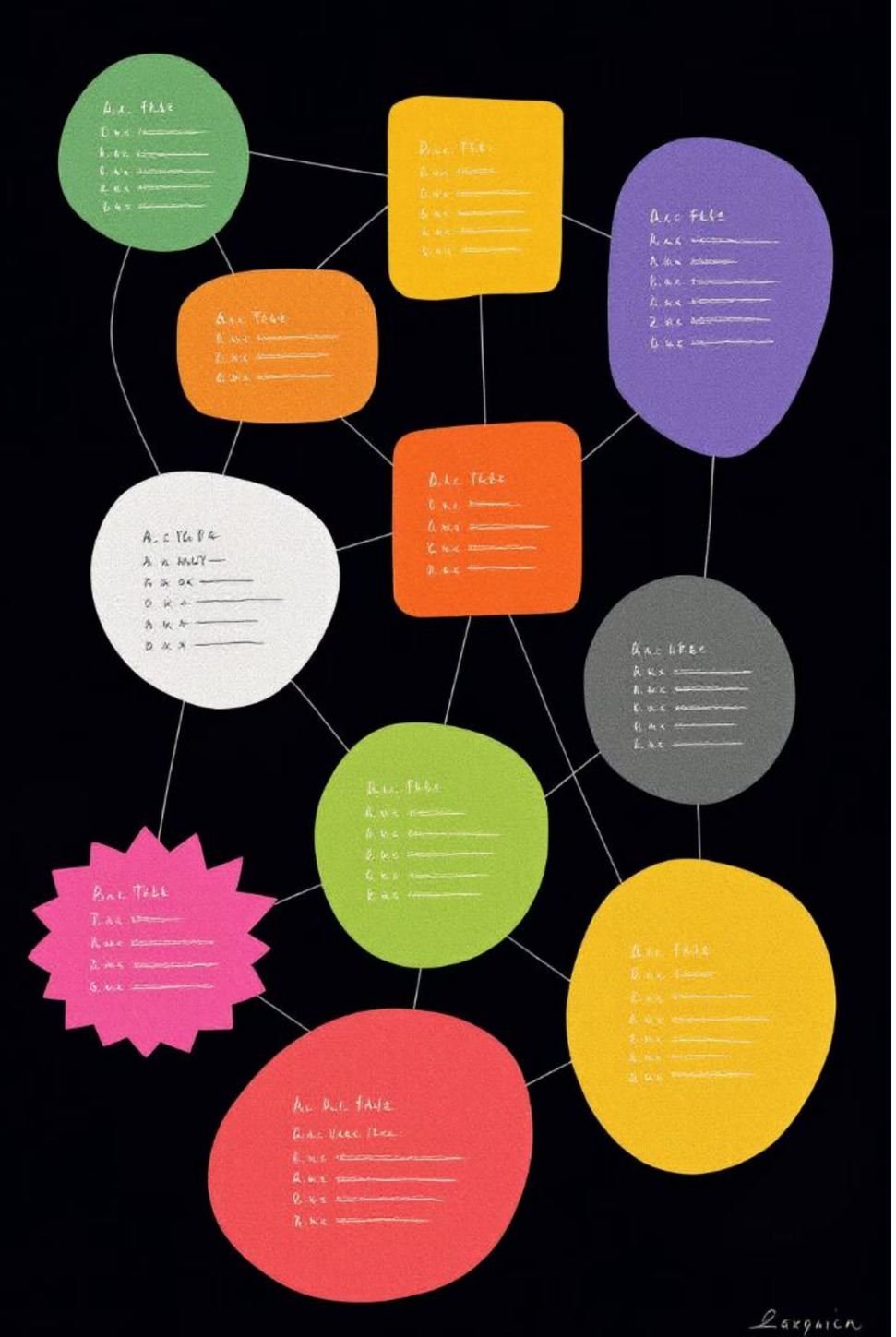
Hibernate utiliza el fichero de mapeo para generar automáticamente las sentencias SQL necesarias al guardar o recuperar datos.



Resumen POJO y fichero de mapeo

Ambos son componentes esenciales en **Hibernate** para garantizar la integración entre el mundo orientado a objetos y las bases de datos relacionales.

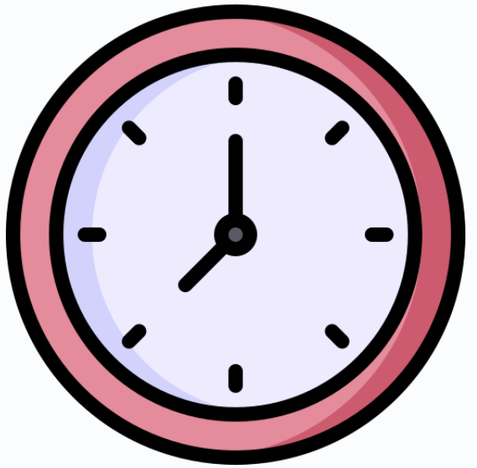


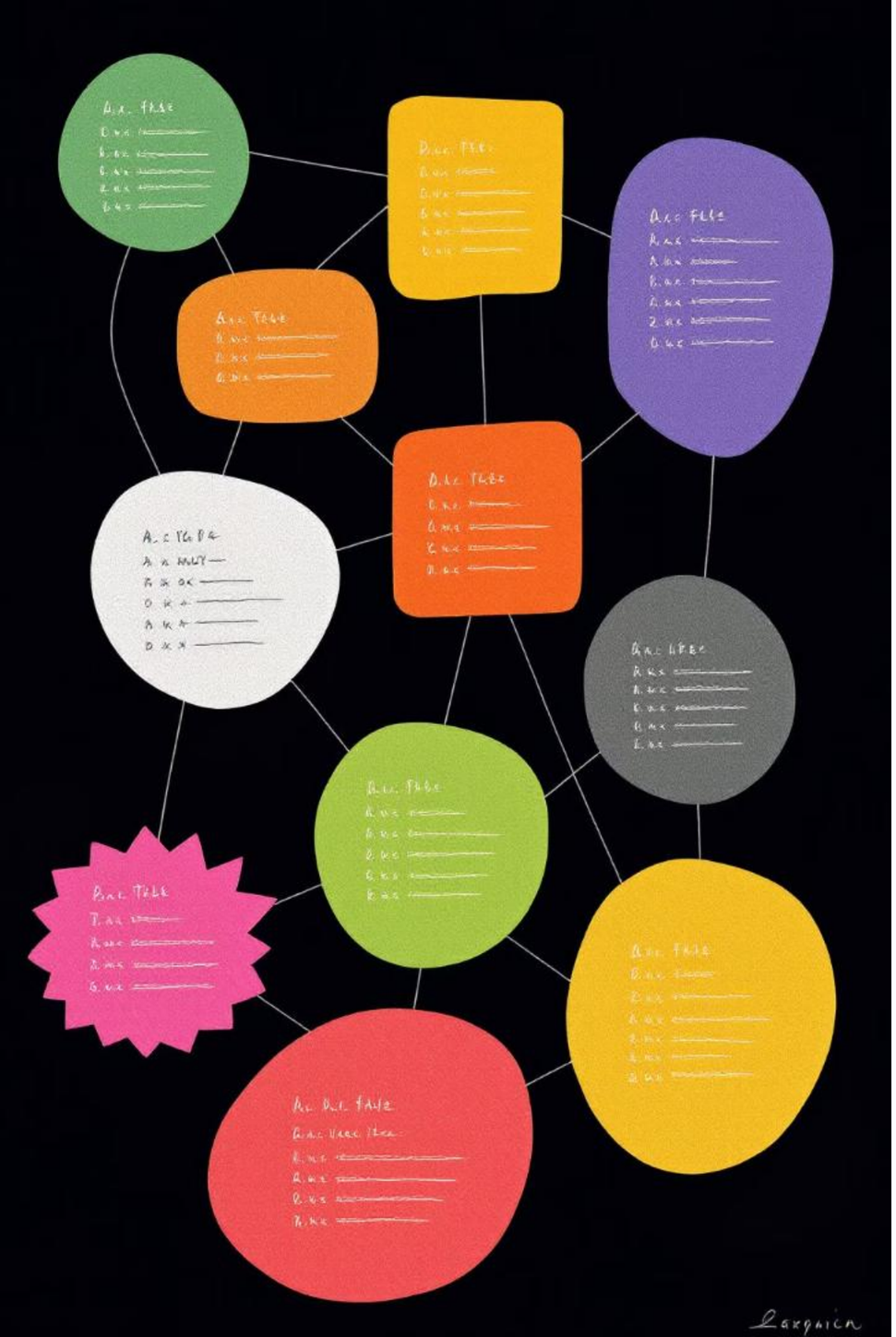


4.4. Mapeo de colecciones, relaciones y herencia

Mapeo de colecciones

- ¿Qué son las colecciones en Java?
- ¿Qué tipo de colecciones son las más comunes en Java?

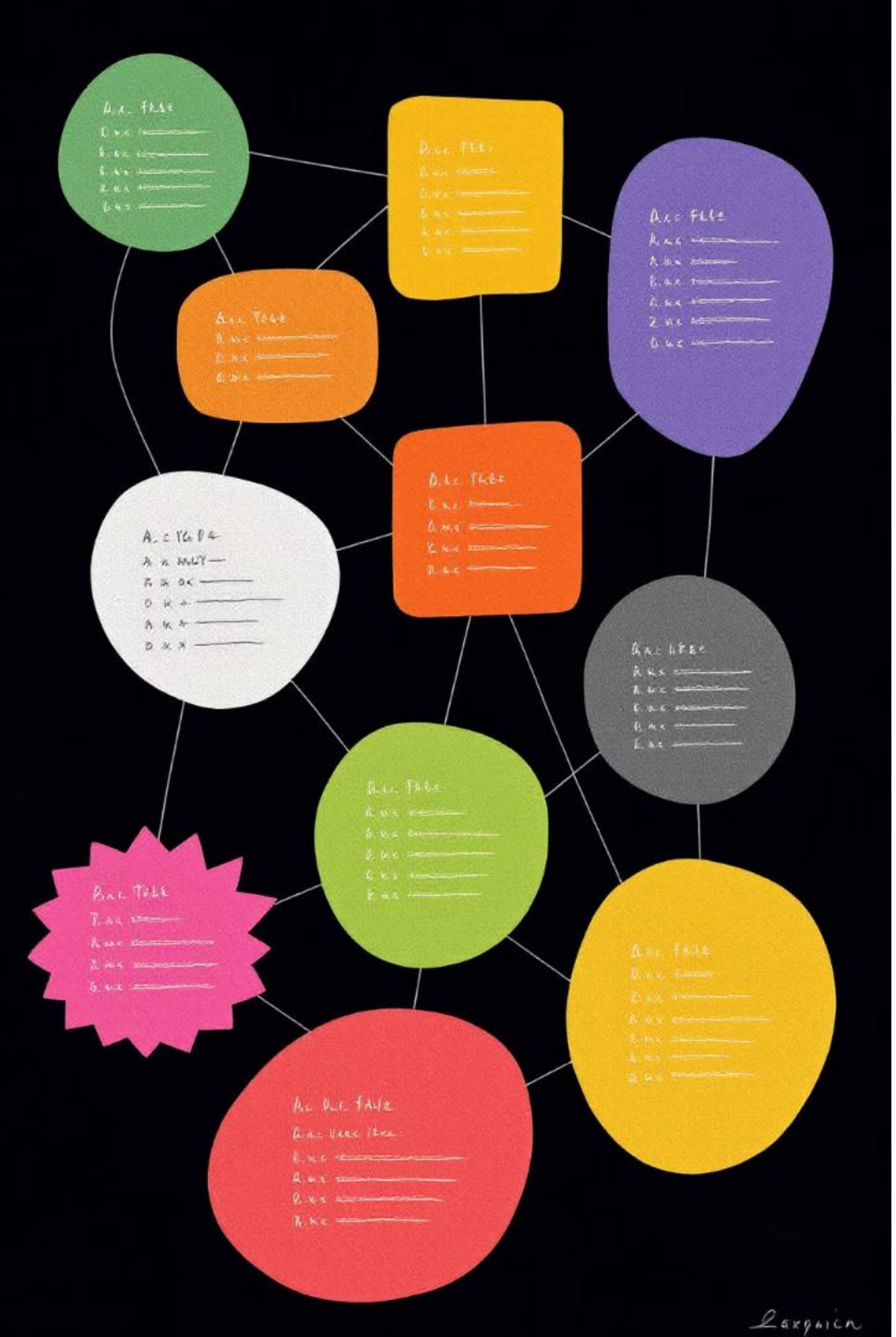




4.4. Mapeo de colecciones, relaciones y herencia

Mapeo de colecciones

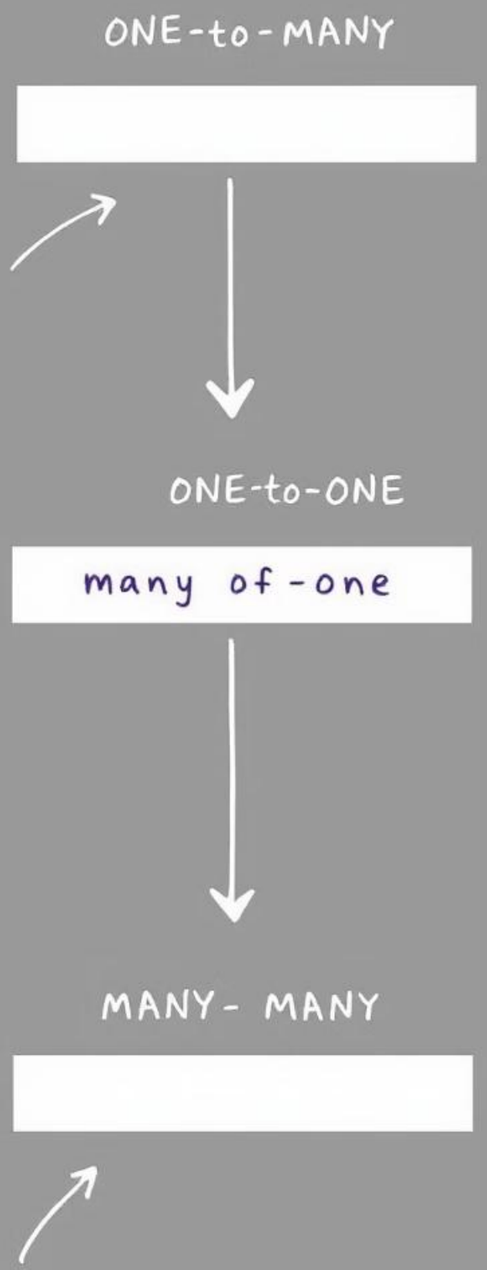
- **¿Qué son las colecciones en Java?**
- Las colecciones en Java son estructuras de datos que permiten almacenar, organizar y manipular un conjunto de elementos relacionados.
- **¿Qué tipo de colecciones son las más comunes en Java?**
  - List: Permite elementos duplicados, y los elementos están ordenados en base al índice. (ejemplo: lista de nombres)
  - Set: No permite elementos duplicados.
  - Map: Almacena pares clave-valor. (ejemplo: nombre y edad asociada)



4.4. Mapeo de colecciones, relaciones y herencia

Mapeo de colecciones

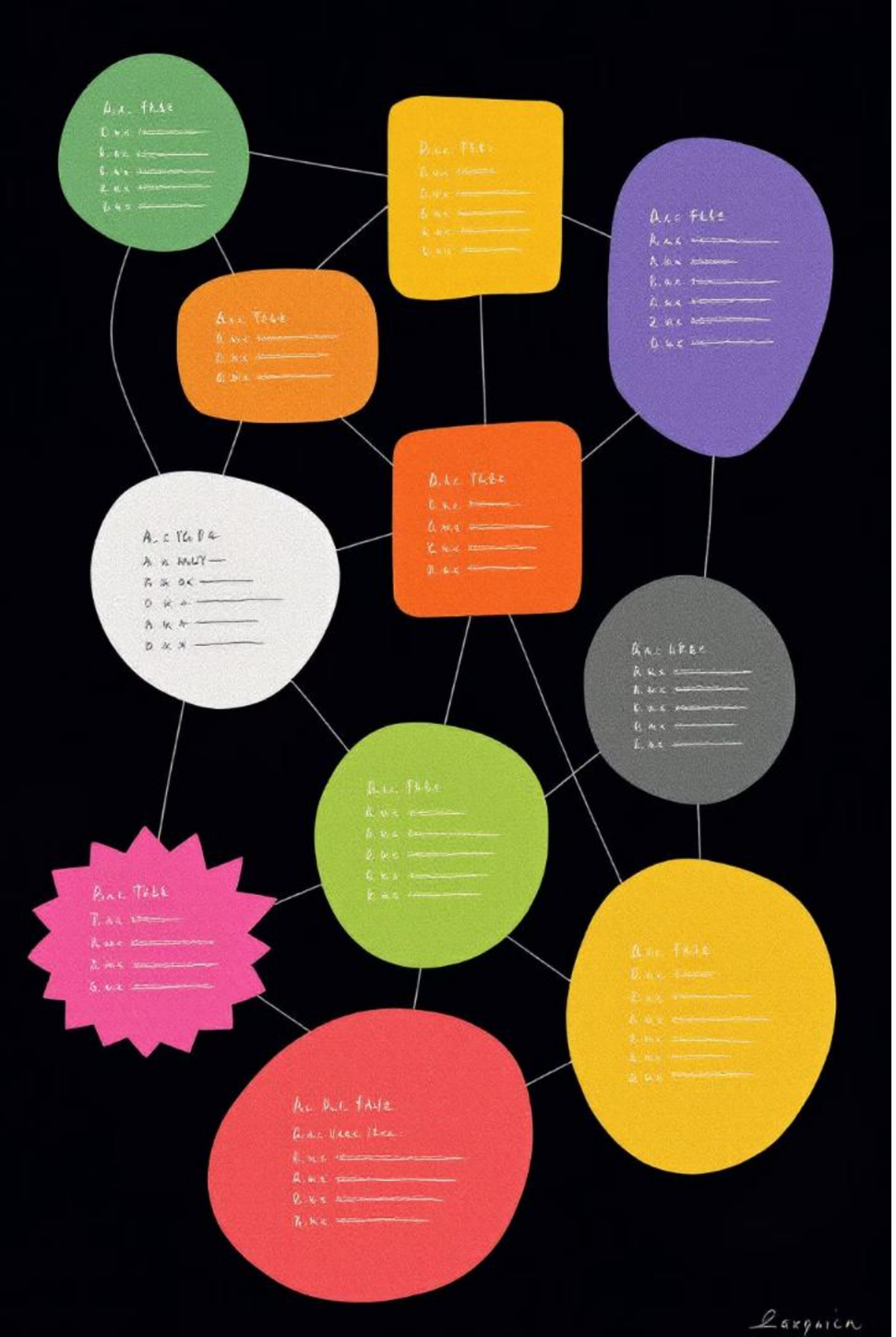
- **Hibernate** admite el mapeo de colecciones como **List, Set o Map** en las clases Java.
- Estas colecciones se pueden vincular con tablas relacionadas.



4.4. Mapeo de colecciones, relaciones y herencia

Relaciones entre tablas

1. **Uno a muchos (one-to-many):** Vincula un objeto con una colección de otros objetos. Por ejemplo, un álbum con sus canciones.
2. **Muchos a uno (many-to-one):** Vincula varios objetos con un solo objeto relacionado. Por ejemplo, varias canciones asociadas a un álbum.
3. **Muchos a muchos (many-to-many):** Utiliza una tabla intermedia para establecer relaciones. Por ejemplo, autores y libros.



4.4. Mapeo de colecciones, relaciones y herencia



Mapeo de herencia

1. **Una tabla por clase:** Todas las clases de la jerarquía comparten la misma tabla.
2. **Una tabla por subclase:** Cada subclase tiene su propia tabla.

Por ejemplo, se puede tener una clase base 'Vehículo' y clases derivadas como 'Coche' y 'Moto' que hereden de ella.

4.5. ¿Qué es una Entidad JPA?

Definición de Entidad: Una Entidad es un POJO (Plain Old Java Object) que el framework JPA tiene la capacidad de gestionar y persistir en una base de datos.

No es solo un objeto en memoria; es la representación viva de una fila en una tabla de la base de datos.

La forma de indicarle a JPA que una clase es una entidad es mediante la anotación `@Entity`.



Diferencias clave

Un POJO simple

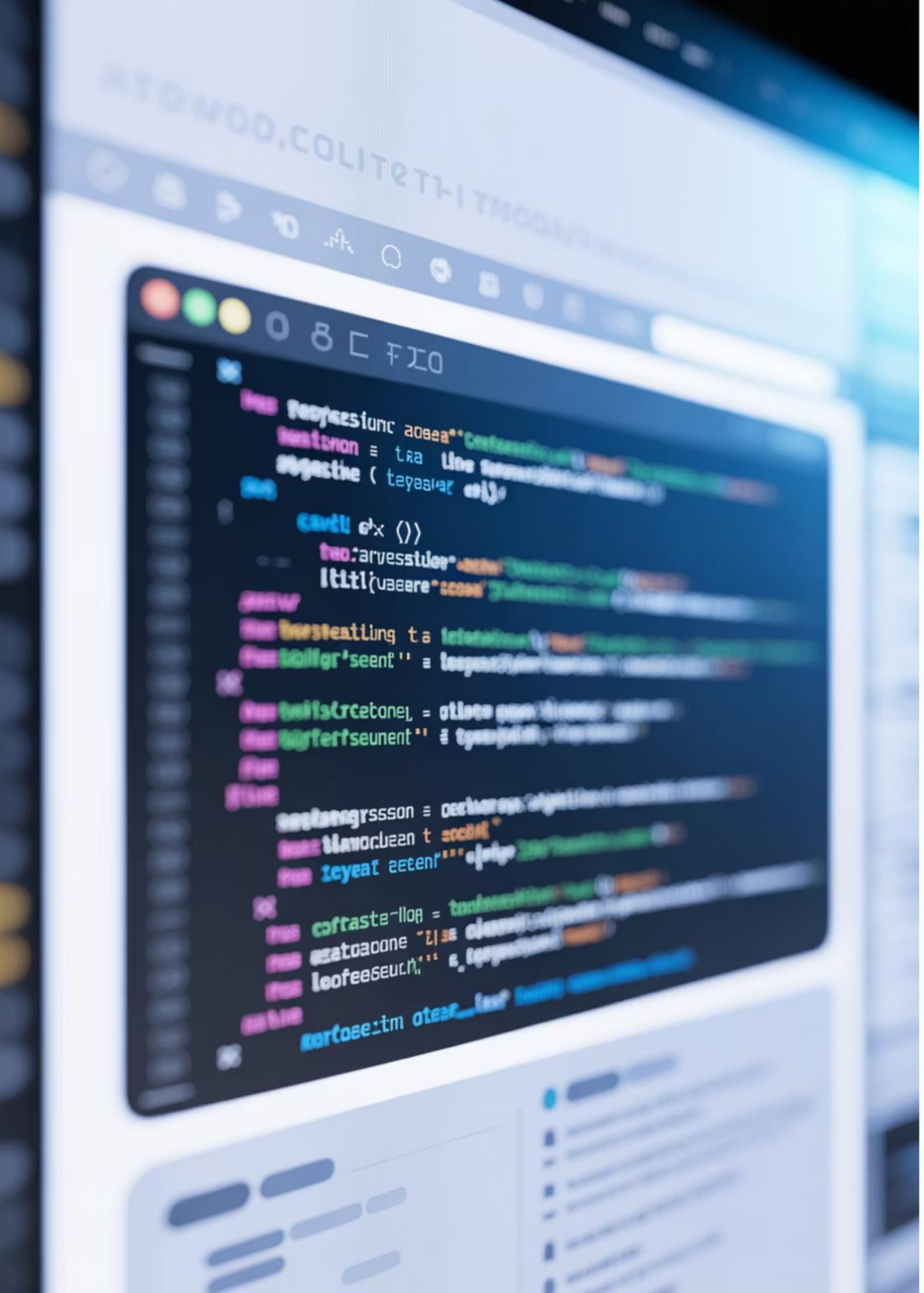
Es un objeto Java normal (getters, setters, atributos). No sabe nada sobre la base de datos ni cómo guardarse. Solo vive en la memoria RAM.

POJO con Mapeo XML (Ej: Hibernate .hbm.xml)

Es el "método antiguo". El POJO sigue estando "limpio" (sin código de persistencia), pero existe un fichero XML externo (.hbm.xml) que actúa como un "manual de instrucciones" para que Hibernate sepa cómo traducir ese objeto a tablas.

Desventaja: El mapeo está separado del código. Es difícil de mantener





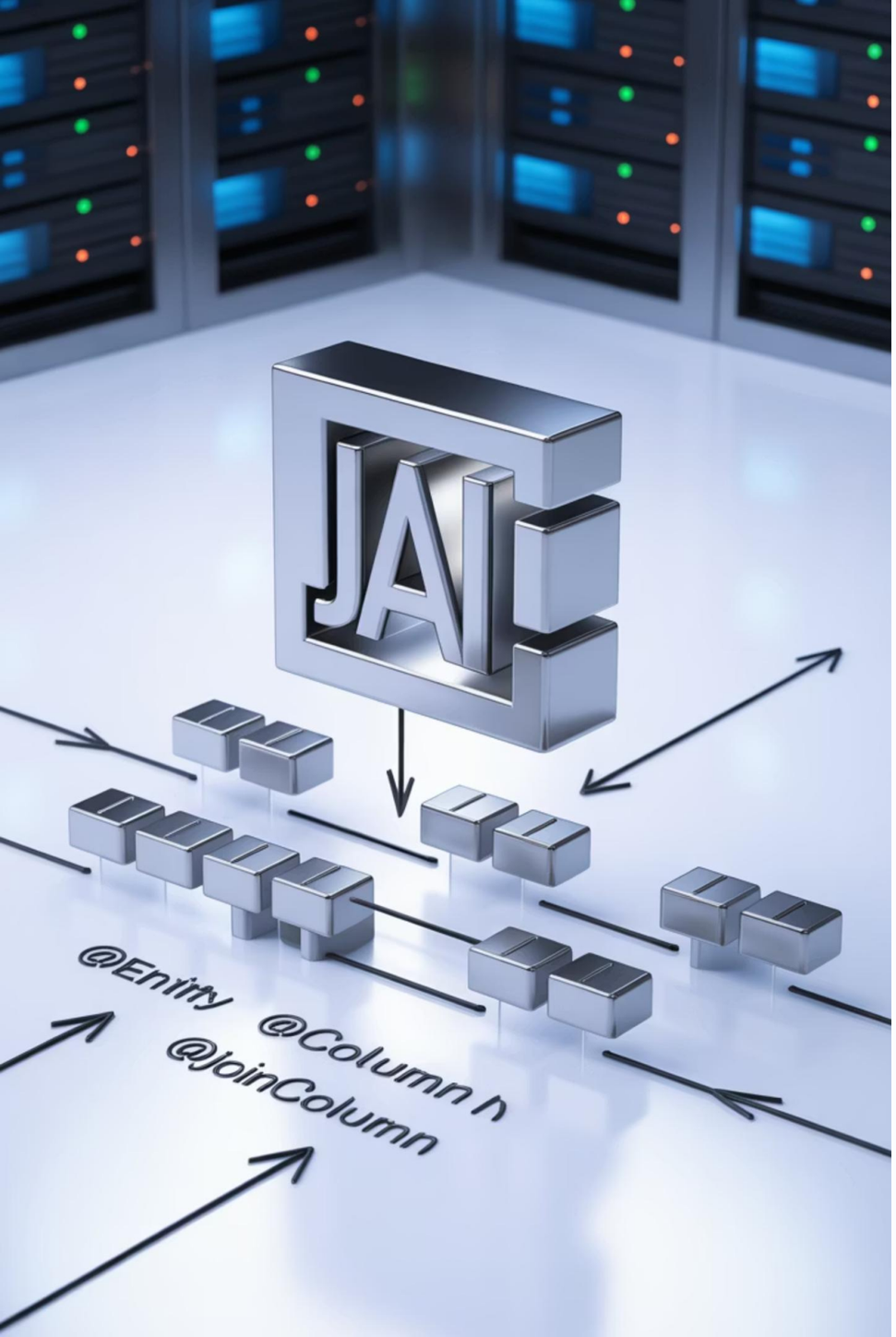
Una Entidad JPA (con anotaciones)

Es el "**método moderno**". Sigue siendo un POJO, pero se autodescribe usando anotaciones (metadatos) dentro de la propia clase Java. El código y las reglas de mapeo viajan juntos.

Ventaja: Mucho más directo, limpio y fácil de mantener.

¿Entidad vs. Clase Persistente?

Son prácticamente sinónimos. Una Entidad JPA es una Clase Persistente. "Clase Persistente" es el término conceptual genérico, y "Entidad" es el término específico que usa JPA para referirse a sus clases persistentes gestionadas.



Anotaciones básicas de Mapeo

JPA utiliza anotaciones para entender cómo mapear una clase Java a una tabla de base de datos.

@Entity

Es la anotación más importante.

Marca la clase como una Entidad que debe ser gestionada por JPA.

Si una clase no tiene `@Entity`, JPA la ignora.

@Id

Marca el atributo que funciona como Clave Primaria (Primary Key) de la tabla.

Toda `@Entity` está obligada a tener un `@Id`.

@Table

Anotación opcional para personalizar el nombre de la tabla.

Si no se usa, JPA asume que la tabla se llama igual que la clase (ej: clase Agentes → tabla AGENTES).

Uso: @Table(name = "agentes_telefonia")

Más anotaciones de Mapeo

@Column

Anotación opcional para personalizar el mapeo de un atributo con una columna.

Uso principal:

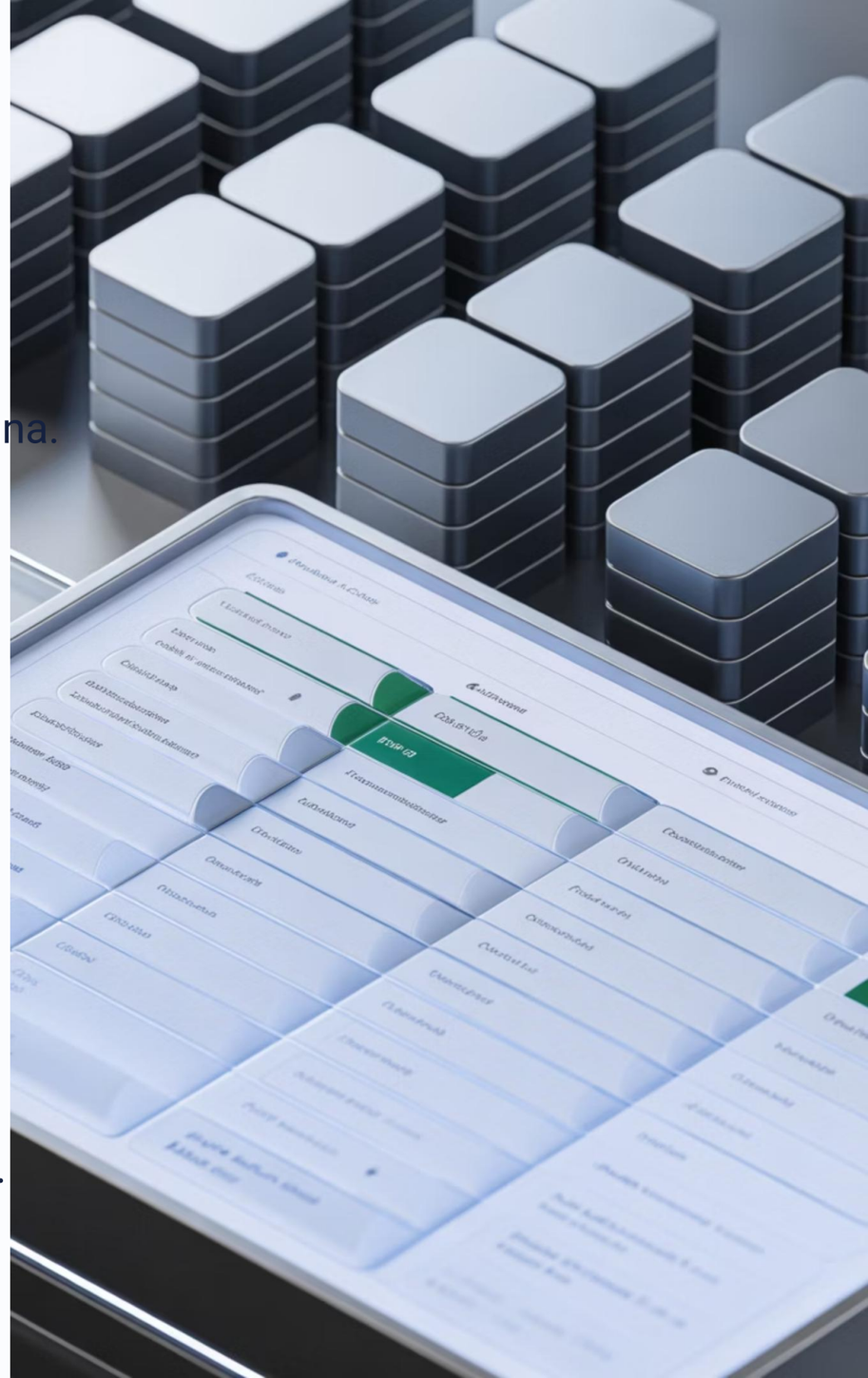
- `name`: Para indicar un nombre de columna diferente (ej: `@Column(name = "nombre_agente")`).
- `length`: Para definir el tamaño (ej: `length = 100` para un VARCHAR(100)).
- `nullable`: Para indicar si la columna no puede ser nula (ej: `nullable = false`).

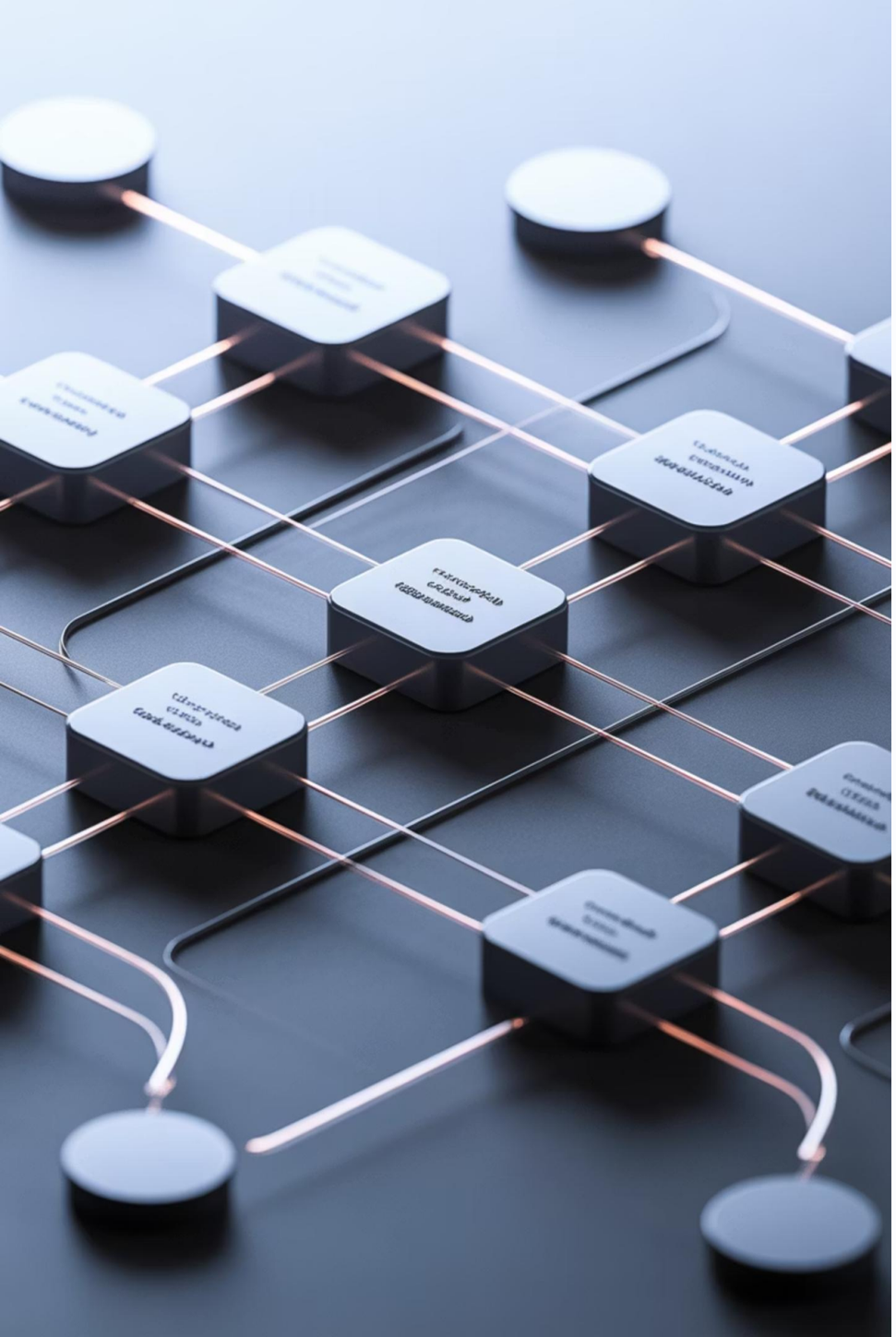
@GeneratedValue

Se puede usar junto a `@Id`.

Indica que el valor del ID es generado automáticamente por la base de datos.

Uso: `@GeneratedValue(strategy = GenerationType.IDENTITY)` (para columnas autoincrementales).





Anotaciones de relaciones (asociaciones)

Estas anotaciones definen cómo se conectan las entidades entre sí (claves foráneas).

@ManyToOne (La más común)

Representa la relación **"Muchos a Uno"**. Es el lado N de la relación 1-N.

Se coloca en la clase "hija".

Ejemplo: Muchas LlamadasEmitidas pertenecen a Una TarjetaTelefonica. La anotación `@ManyToOne` va en la entidad LlamadasEmitidas, sobre el atributo tarjeta.

JPA automáticamente crea la columna de clave foránea (SIM_LLAMANTE en tu caso).

@OneToMany (El lado opuesto)

Representa la relación **"Uno a Muchos"**. Es el lado 1 de la relación 1-N.

Se coloca en la clase "padre".

Ejemplo: Una TarjetaTelefonica tiene Muchas LlamadasEmitidas. La anotación `@OneToMany` va en la entidad TarjetaTelefonica, sobre el atributo `List<LlamadasEmitidas> llamadas`.



Parámetros clave de las relaciones

fetch (Modo de carga)

Controla cuándo JPA carga los datos de la relación.

- **FetchType.LAZY (Perezosa):** Opción recomendada. JPA no carga las entidades relacionadas hasta que se acceden por primera vez (ej: `tarjeta.getLlamadas()`). Evita consultas pesadas.
- **FetchType.EAGER (Ansiosa):** JPA carga la entidad y todas sus relaciones en la misma consulta. Puede ser muy lento e ineficiente.

Regla: `@OneToMany` es LAZY por defecto. `@ManyToOne` es EAGER por defecto. (¡Cuidado con esto!).

Cascade y MappedBy

cascade (Propagación en cascada)

Define si las operaciones (como guardar o borrar) se propagan del padre al hijo.
Ejemplo: `cascade = CascadeType.ALL` en un `@OneToMany` significa que si borras la TarjetaTelefonica (padre), JPA borrará automáticamente todas sus LlamadasEmitidas (hijas).

mappedBy (el "dueño" de la relación)

Solo se usa en `@OneToMany`.
Es obligatorio en relaciones bidireccionales.

Ejemplo: `mappedBy = "tarjeta"` en `@OneToMany`.

Le dice a JPA: "No crees una clave foránea para esta relación; la relación ya está gestionada (mappedBy) por el atributo 'tarjeta' de la otra entidad (LlamadasEmitidas)".



5. GESTIÓN DE SESIONES CON JPA

EntityManagerFactory



Análogo a SessionFactory en Hibernate

¿Qué es?
Es una "fábrica" pesada y *thread-safe*.

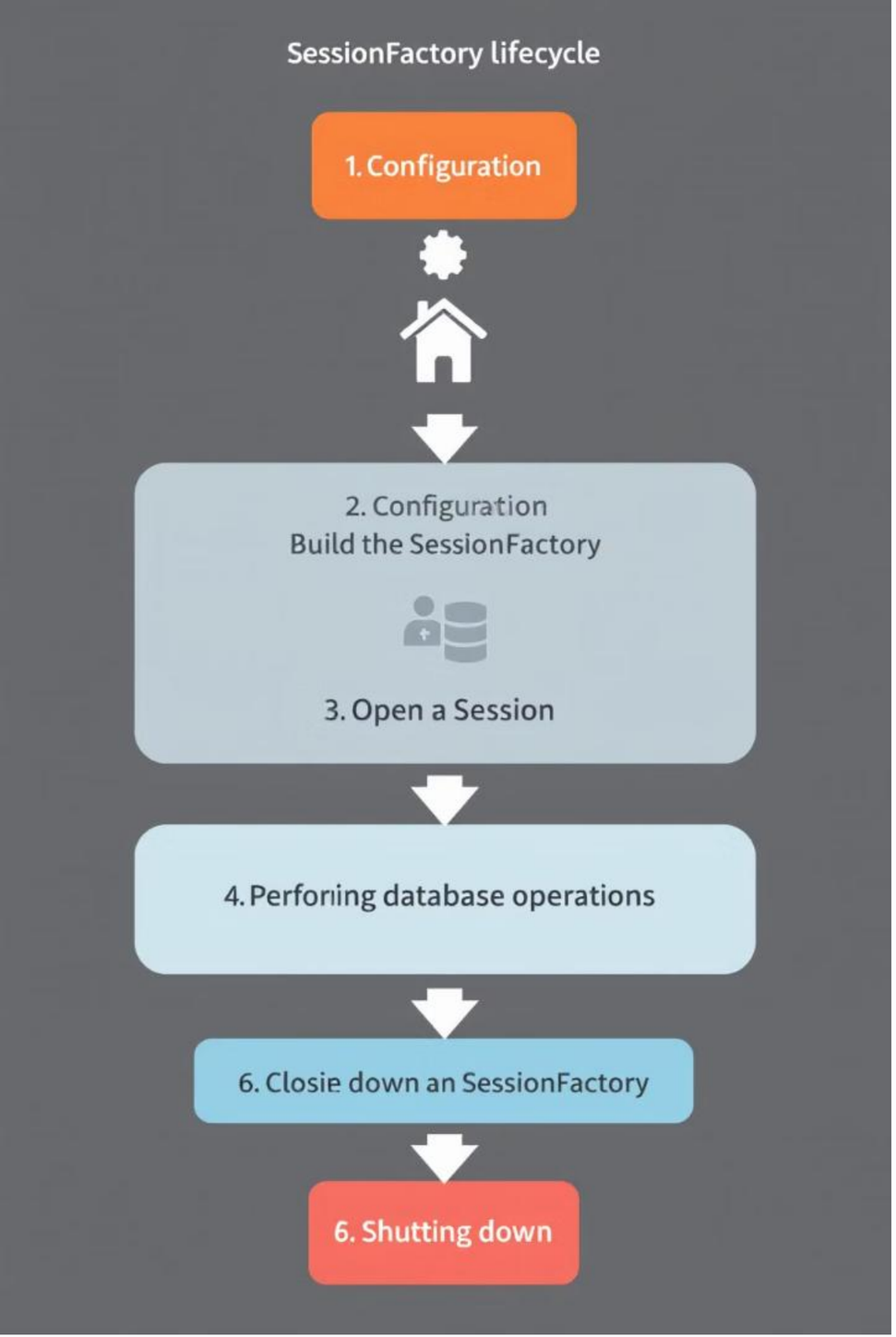
Creación
Se crea **UNA SOLA VEZ** al inicio de la aplicación.

Configuración
Lee los datos del fichero META-INF/persistence.xml.

Propósito
Su única misión es crear EntityManager.

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("nombre-unidad-persistencia");
```





La "Session" de JPA: EntityManager

Características Principales

Análogo a la Session de Hibernate.

¿Qué es? Es el objeto principal para interactuar con la base de datos.

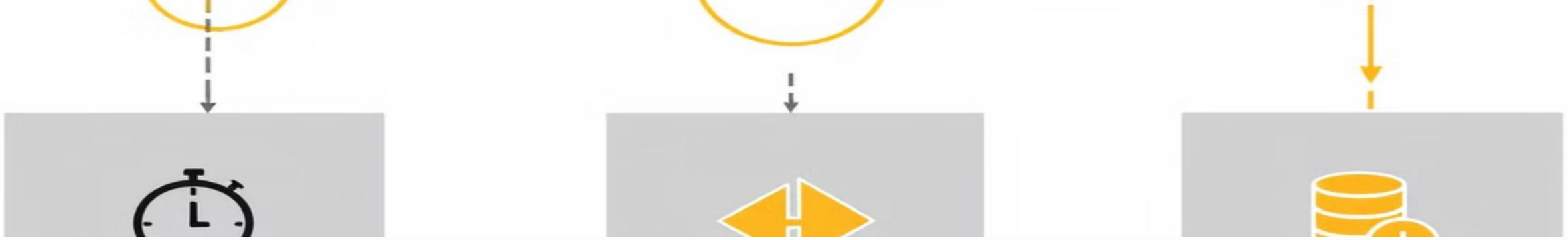
Características: Es ligero y **NO** es *thread-safe*. Está diseñado para una "unidad de trabajo" corta.

Contexto de Persistencia

Gestiona la caché de primer nivel y el seguimiento de las entidades.

Gestión

```
EntityManager em = emf.createEntityManager();
…
em.close(); // ¡Siempre cerrar al terminar!
```



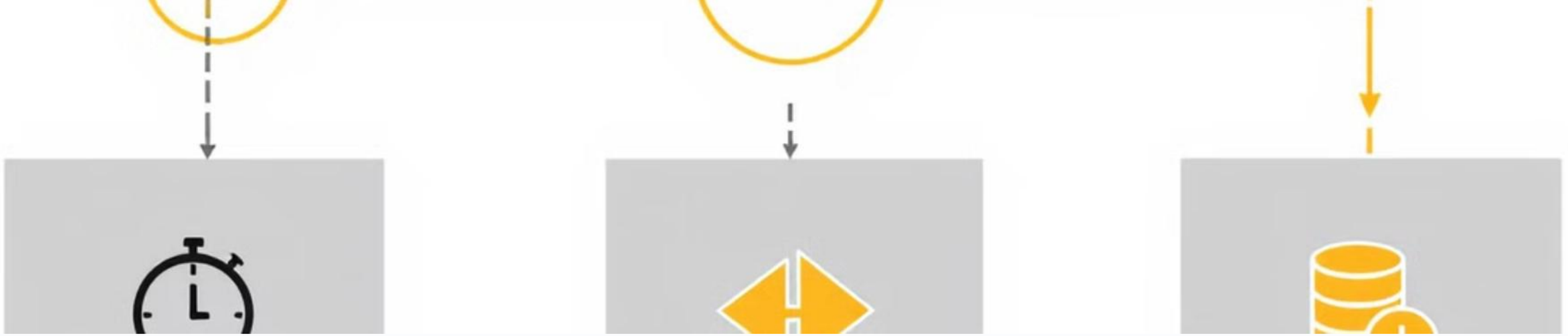
Patrones

```java
// PATRON PARA CONSULTAS (sin transaccion)
public void consultarDatos() {
    EntityManager em = emf.createEntityManager();
    try {
        // Solo lectura - NO necesita transacción
        MiEntidad entidad = em.find(MiEntidad.class, id);
        // usar entidad...
    } finally {
        em.close();
    }
}
```

```java
// PATRON PARA MODIFICACIONES (con transaccion)
public void modificarDatos() {
    EntityManager em = emf.createEntityManager();
    EntityTransaction tx = null;
    try {
        tx = em.getTransaction();
        tx.begin(); // ← NECESARIO

        // Operaciones de escritura (INSERT/UPDATE/DELETE)

        tx.commit(); // ← NECESARIO
    } catch (Exception e) {
        if (tx != null && tx.isActive()) {
            tx.rollback();
        }
    } finally {
        em.close();
    }
}
```



5.2.4. Ventajas de Utilizar EntityManager

1

Control del ciclo de vida

JPA maneja automáticamente los estados de los objetos.

2

Gestión Eficiente

Reduce consultas innecesarias utilizando la caché de primer nivel.

3

Contexto de persistencia

Establece un alcance claro para las operaciones de persistencia.

6. Operaciones fundamentales de persistencia

Métodos CRUD con JPA

CREATE → persist()

01

Método

```
em.persist(objeto);
```

02

Propósito

Convierte un objeto *nuevo* (transient) en *gestionado* (managed).

03

Funcionamiento

El objeto se añade al contexto de persistencia. El INSERT SQL no se ejecuta de inmediato, sino que se "agenda" para el commit de la transacción.

04

Requisito

Debe ejecutarse **obligatoriamente** dentro de una transacción activa.

READ (Buscar) → find()

Método

```
MiClase obj = em.find(MiClase.class, id);
```

Análogo a session.get() de Hibernate.



Propósito

Cargar una entidad por su Clave Primaria.

Funcionamiento

1. Busca en el contexto de persistencia (caché 1er nivel).
2. Si no está, ejecuta un SELECT a la base de datos.

Resultado: Devuelve la entidad gestionada (managed) o null si no se encuentra.

READ (Referencia) → getReference()

1

Método

```
MiClase obj =
em.getReference(MiClase.class, id);
```

2

Propósito

Obtener una referencia *proxy* (un objeto "vacío" solo con el ID).

3

Funcionamiento

No va a la base de datos (carga diferida o *lazy*). Asume que la entidad existe.

Análogo a session.load() de Hibernate.

¡Cuidado! Si la entidad no existe, lanzará EntityNotFoundException cuando intentes usar el objeto (no al llamar a getReference).

UPDATE → ¡Automático! (Dirty Checking)

¡No hay un método em.update() en JPA!



Propósito

- Modificar una entidad existente.

Funcionamiento (Dirty Checking)

Cargar

Obtén una entidad *gestionada* (ej. con em.find()).

Modificar

Llama a sus métodos *setters* (ej. obj.setNombre("Nuevo")).

Commit

Al hacer commit de la transacción, JPA compara el estado actual del objeto con su estado original. Si detecta cambios ("dirty"), ejecuta el UPDATE SQL automáticamente.

DELETE → remove()

Método

```
em.remove(objeto);
```

Análogo a session.delete() de Hibernate.

Propósito

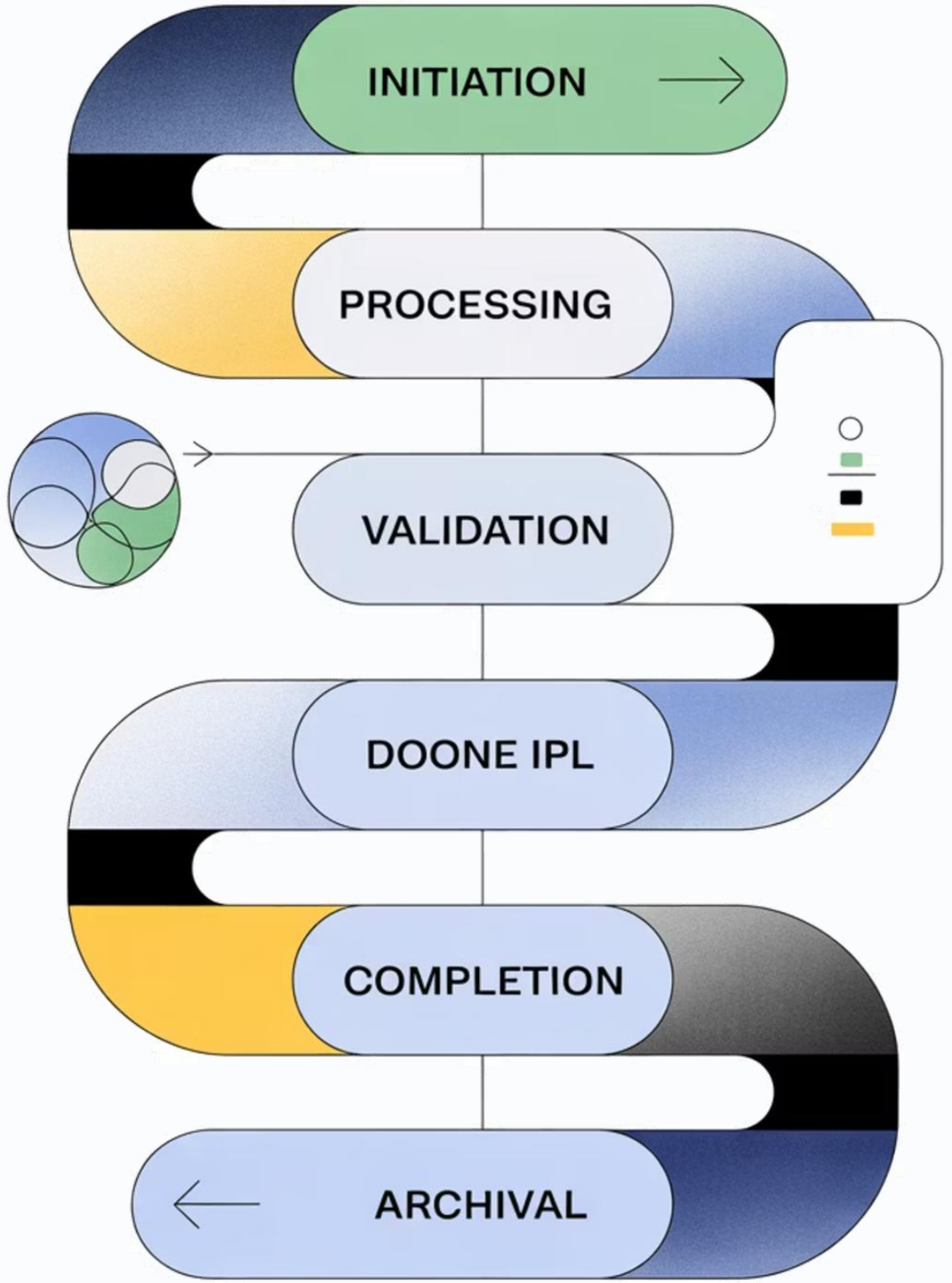
Marcar una entidad para su eliminación.

Funcionamiento

- El objeto debe estar *gestionado* (managed) para poder ser borrado.
- **Flujo común:** `em.remove(em.find(MiClase.class, id));`
- El DELETE SQL se "agenda" para ejecutarse en el commit.



7. Estados de los Objetos y cambios de estado





2. Características y herramientas ORM

2.3. Hibernate: estados de los objetos

Objetos transitorios (transient)

Son objetos que existen únicamente en memoria y no están vinculados a una base de datos.

Ejemplo: Un objeto creado en la aplicación pero aún no persistido.

Objetos persistentes

Son objetos que están **sincronizados con una base de datos relacional y vinculados a una sesión (Session)**. Hibernate permite convertir un objeto transitorio en persistente mediante métodos como save() o persist(). También proporciona una interfaz para realizar consultas mediante su lenguaje de consultas propio, HQL (Hibernate Query Language).

Los 4 estados de una entidad JPA

New / Transient (Nuevo)

- Creado con new (Ej: new Persona();).
- JPA no lo conoce. No está en el contexto de persistencia.

Managed / Persistent (Gestionado)

- El objeto está asociado al contexto de persistencia.
- Obtenido tras un find(), getReference() o persist().
- ¡Los cambios se sincronizarán automáticamente!

Detached (Separado / Disociado)

- El objeto *estuvo* gestionado, pero el EntityManager que lo controlaba se cerró (o se usó em.detach()).
- Los cambios *no* se sincronizan.

Removed (Eliminado)

- Un objeto *gestionado* que ha sido pasado a em.remove().
- Está "agendado" para borrarse en el commit.

Transiciones de estado (el ciclo de vida)

New → Managed

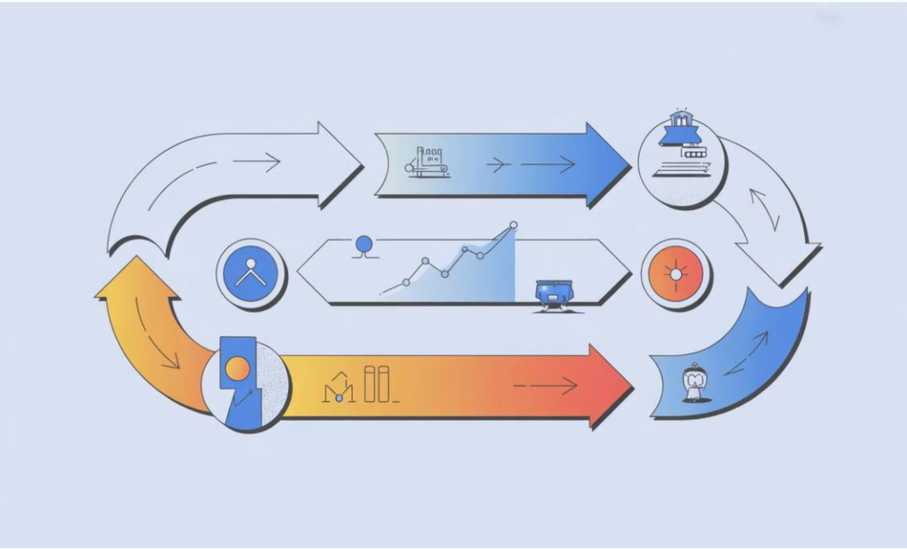
`em.persist(objeto);`

(Base de Datos) → Managed

```
MiClase obj = em.find(...);
```

Managed → Removed

`em.remove(objeto);`



Managed → Detached

`em.close();` (Todos los objetos que gestionaba quedan separados).

`em.detach(objeto);` (Separa un objeto específico).

• Volver de detached a managed → merge()

El problema

Tienes un objeto *Detached* (ej. viene de una petición web) y quieres guardar sus cambios.

La solución

merge()

Método

```
MiClase gestionada =
em.merge(objetoDetached);
```

Funcionamiento (¡Importante!)

1. merge() recibe el objeto Detached.
2. Carga la versión actual (Managed) de la BD (o crea una nueva si no existe).
3. Copia los cambios del objeto Detached al Managed.
4. **Devuelve** la instancia Managed.

> **Error común:** Seguir usando objetoDetached tras el merge. ¡Debes usar la gestionada que te devuelve el método!

8. Consultas SQL tradicionales



SQL Nativo: createNativeQuery()

¿Por qué usarlo?

- Consultas muy complejas (ej. WITH RECURSIVE).
- Aprovechar funciones específicas de tu motor de BD (ej. JSON_B en PostgreSQL).

Desventaja

Pierdes la portabilidad entre bases de datos.

Método

`em.createNativeQuery(string_sql_puro);`

Ejemplo (devuelve Object[])

```
Query q = em.createNativeQuery("SELECT p.ID, p.NOMBRE FROM PERSONA p");List<Object[]> resultados = q.getResultList();
```

SQL nativo con mapeo a entidad

El objetivo

Ejecutar SQL puro, pero recibir una lista de tus objetos de Entidad.



Método

```
em.createNativeQuery(string_sql_puro, TuClase.class);
```

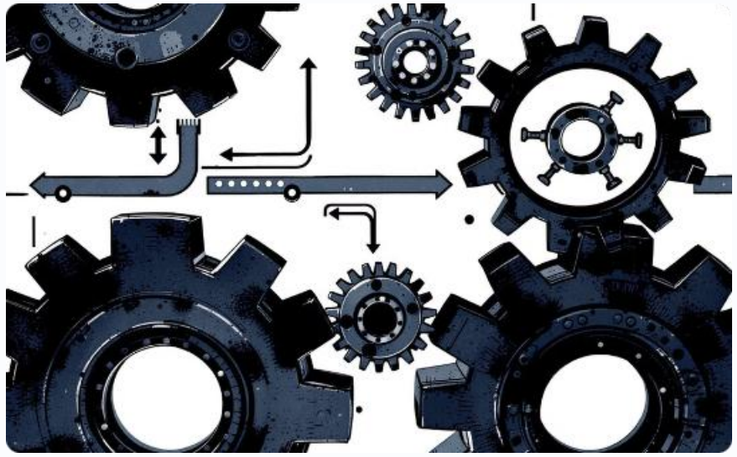
Ejemplo

```
String sql = "SELECT * FROM PERSONA WHERE EDAD > 18";
TypedQuery<Persona> q = em.createNativeQuery(sql, Persona.class);
List<Persona> personas = q.getResultList();
```

Requisito: El SELECT debe devolver todas las columnas que @Entity espera.

Ventajas y desventajas del enfoque SQL tradicional







Consultas personalizadas

El enfoque SQL tradicional permite usar consultas complejas o específicas del motor de base de datos. Útil cuando queremos aprovechar las características avanzadas de SQL no soportadas por HQL

Migración de aplicaciones

Facilita la migración de aplicaciones que ya usan SQL nativo.

Dependencia del motor

Las consultas SQL son dependientes del motor de base de datos, lo que puede dificultar la portabilidad. No aprovechan completamente las capacidades orientadas a objetos de Hibernate.

Consultas JPQL

JPQL (Java Persistence Query Language)

El "SQL" estándar de JPA	Orientado a Objetos	Ventaja
(Análogo directo a HQL).	Se consulta sobre **Clases** (Entidades) y **Atributos**, NO sobre Tablas y Columnas.	Totalmente portable. JPA lo traduce al SQL específico del motor de BD que estés usando.

Sintaxis

```
SELECT p FROM Persona p
```

(¡Observa! Persona es el nombre de la *Clase*, no de la *tabla*).

Ejecutando consultas JPQL

1. Crear la Query (TypedQuery)

```
String jpql = "SELECT p FROM Persona p WHERE p.edad > :edadParam";
TypedQuery<Persona> query = em.createQuery(jpql, Persona.class);
```

2. Asignar Parámetros (¡siempre!)

```
query.setParameter("edadParam", 18);
```

(Evita la Inyección SQL y mejora el rendimiento de la caché de queries).

3. Obtener resultados

```
List<Persona> lista = query.getResultList(); // Si esperas variosPersona p =
query.getSingleResult(); // Si esperas uno solo
```

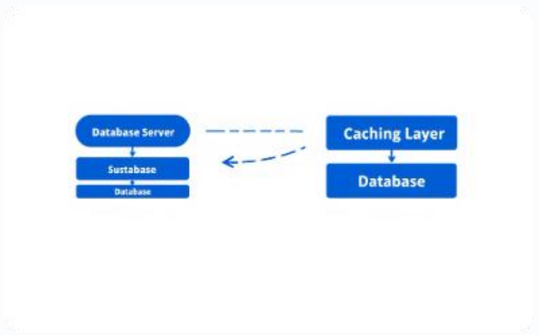
9. Transacciones y ACID

¿Qué es? Una unidad de trabajo que debe ser **ACID**.



Atomicidad

Garantiza que todas las operaciones dentro de una transacción se completen como una unidad.



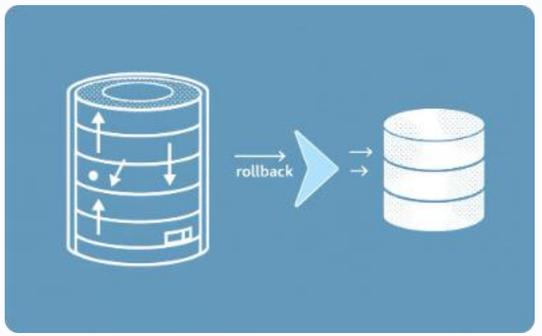
Uso eficiente de recursos

La sesión actúa como una **caché de primer nivel**, minimizando las interacciones con la base de datos.



Facilidad de uso

JPA **simplifica la gestión de transacciones** con su API y su integración con frameworks.



Manejo de errores

Las transacciones permiten **revertir cambios en caso de errores**, manteniendo la coherencia de los datos.

> **Regla de Oro en JPA:** Toda operación de *escritura* (persist, merge, remove) **DEBE** ocurrir dentro de una transacción.